

Índice general

Capítulo 1. Introducción	2
1.1 Fundamento del sistema eléctrico trifásico	4
1.2 Importancia del análisis de corrientes trifásicas	5
Capítulo 2. Metodología	7
2.1 Fundamentos de estadística	7
2.2 Medidas de tendencia central	10
2.3 Outliers (Valores atípicos)	12
2.4 Procesamiento de datos	13
2.5 Análisis exploratorio	14
2.6 Visualización	15
Capítulo 3. Resultados	17
Capítulo 4. Conclusión	23
Apéndice	25

Introducción

En el contexto de las ciudades modernas, el funcionamiento cotidiano de la vida urbana, desde el alumbrado público hasta el uso de electrodomésticos, transporte eléctrico y sistemas de comunicación, depende de forma directa de una infraestructura eléctrica robusta y bien organizada. Esta infraestructura está compuesta por diversos elementos, entre los que destacan los circuitos eléctricos de distribución y las subestaciones eléctricas. Ambos conforman una red compleja que asegura el suministro confiable, eficiente y seguro de la energía eléctrica a los usuarios finales.

Una red eléctrica urbana es un sistema interconectado que distribuye energía eléctrica desde los puntos de transformación hasta los consumidores. Su papel en la ciudad es esencial, ya que posibilita el funcionamiento de hogares, oficinas, industrias, hospitales y todo tipo de infraestructura urbana. Esta red se divide generalmente en niveles de tensión: alta tensión, que transporta la electricidad desde plantas generadoras o subestaciones primarias hasta subestaciones secundarias; media tensión, que lleva la energía desde subestaciones secundarias a zonas más acotadas dentro de la ciudad; y finalmente, baja tensión, que llega directamente a los usuarios (Short, 2014). Una representación gráfica se puede ver en la Figura 1

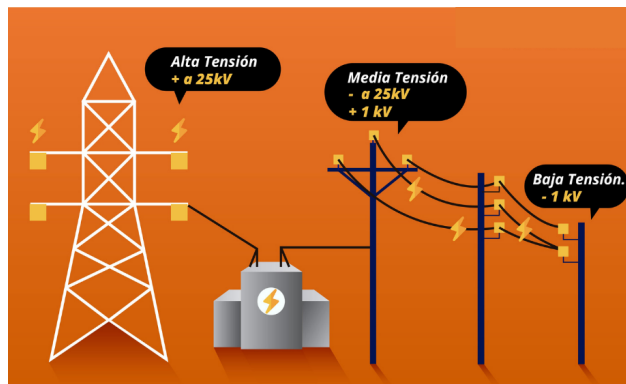


Figura 1: Niveles de tensión en redes eléctricas. Alta tensión ($\geq 25 \text{ kV}$) para transporte a larga distancia, Media tensión ($1 \text{ kV} - 25 \text{ kV}$) para distribución urbana, y Baja tensión ($< 1 \text{ kV}$) para consumo final.

Este sistema no solo debe ser capaz de transportar grandes cantidades de energía, sino también de hacerlo con fiabilidad y seguridad. Para ello, es indispensable una buena configuración de circuitos y un diseño adecuado de subestaciones, de forma que se pueda responder ante fallas, evitar sobrecargas, aislar zonas en mantenimiento y garantizar la continuidad del

servicio eléctrico.

Dentro de esta infraestructura, las subestaciones eléctricas tienen una función estratégica. Actúan como nodos intermedios donde se transforma la tensión de la energía eléctrica y se distribuye a través de diferentes circuitos. Su diseño está pensado no solo para cumplir esta tarea de transformación, sino también para proteger la red y dividirla en zonas controlables.

Por ello, en las ciudades no basta con una sola subestación. Cada subestación alimenta varios circuitos, y cada circuito, a su vez, abastece a una zona específica de la ciudad. Esta configuración permite mantener la estabilidad del sistema, así como aislar problemas en sectores específicos sin comprometer el funcionamiento total de la red.

Uno de los aspectos más importantes del diseño de las redes eléctricas urbanas es la configuración de los circuitos eléctricos. Dependiendo de las necesidades urbanas, se pueden emplear configuraciones radiales, malladas o en anillo. En una configuración radial (ver panel derecho de la Figura 2), cada circuito parte de una subestación y sigue una única trayectoria hacia los usuarios. Es sencilla y económica, pero poco flexible ante fallas. En una configuración mallada como la que se muestra en el panel central de la Figura 2, varios puntos están interconectados formando una malla, lo cual permite múltiples trayectorias para la energía. Esto ofrece mayor confiabilidad, pero requiere más inversión y sistemas de control más complejos. Por otro lado, la configuración en anillo (ver panel izquierdo de la Figura 2) es un intermedio entre ambas: los circuitos están conectados en un bucle cerrado. Si se interrumpe un punto, la energía puede llegar por el lado opuesto del anillo, lo cual asegura mayor continuidad del servicio (Short, 2014).

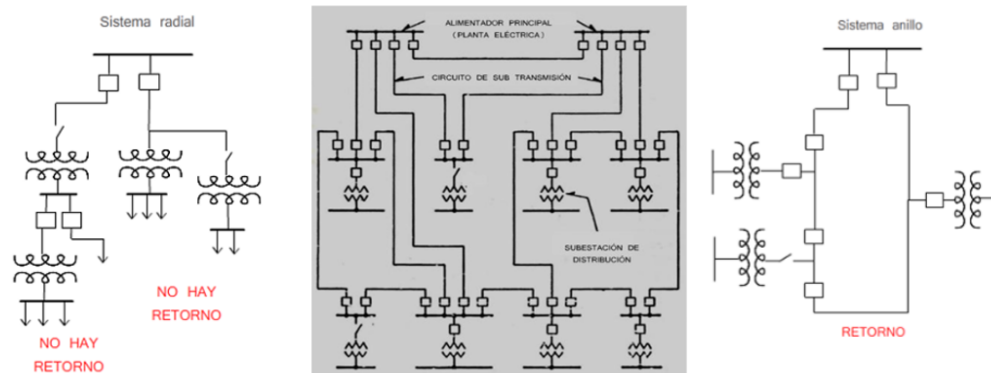


Figura 2: Sistemas de distribución eléctrica. Sistema radial (panel derecho). Sistema simple con una única trayectoria desde la subestación hasta la carga, distribuido en forma de ramas, con menor confiabilidad ante fallas. Sistema de mallas (panel central). Subtransmisión que alimenta desde dos plantas a cualquier subestación, garantizando continuidad del servicio para instalaciones críticas. Sistema de anillo (panel izquierdo). Sistema que forma un ciclo completo desde la subestación, alimentando desde dos extremos y permitiendo aislar secciones en caso de falla, mejorando la confiabilidad.

Dividir el sistema eléctrico urbano es una estrategia de control y mantenimiento. Gracias

a esta división, cuando ocurre una falla o es necesario hacer trabajos de mantenimiento, se puede desconectar solamente un sector limitado de la red sin afectar al resto de la ciudad. Además, permite monitorear el flujo de energía en cada circuito, detectar sobrecargas, planificar ampliaciones y optimizar la eficiencia del servicio.

1.1 Fundamentos del sistema eléctrico trifásico

Un sistema trifásico es una forma de generar, transmitir y distribuir energía eléctrica que utiliza tres corrientes alternas de igual frecuencia y amplitud, pero se desfasa 120 grados entre sí. Es la forma más común y eficiente de suministrar electricidad en redes de media y alta tensión, especialmente en aplicaciones industriales, comerciales e infraestructura urbana.

En los sistemas eléctricos trifásicos de distribución, como los que operan típicamente en media tensión (600 V a 35 kV), es fundamental realizar un registro detallado de las corrientes de cada fase (I_a , I_b , I_c) así como de la corriente de neutro, I_n . Esto se debe a que la medición individual de las corrientes por fase permite evaluar con precisión el comportamiento del sistema eléctrico bajo condiciones normales y anómalas. Las corrientes I_a , I_b e I_c representan las magnitudes de corriente que circulan por cada una de las fases A, B y C, respectivamente, mientras que la corriente I_n corresponde a la que circula por el conductor neutro, el cual generalmente está conectado a tierra en varios puntos a lo largo de su recorrido, con el fin de garantizar la seguridad y facilitar la detección de corrientes residuales. El análisis de estas corrientes sirve para identificar desequilibrios en la carga, pues en un sistema balanceado la suma vectorial de todas las cargas debería ser cero; cualquier desviación indica presencia de corriente en el neutro que puede estar asociada a cargas monofásicas desbalanceadas o a fallas fase-tierra ([Bergen and Vittal, 2000](#)).

Además, el registro de estas variables permite la aplicación del método de componentes simétricas, una técnica utilizada en ingeniería eléctrica para simplificar el análisis de sistemas trifásicos desequilibrados. Este método transforma las corrientes de fase en tres componentes: la secuencia positiva (I_1), que representa la corriente balanceada en condiciones normales de operación; la secuencia negativa (I_2), asociada a desequilibrios en las fases; y la secuencia cero (I_0), que refleja las corrientes residuales comunes en fallas a tierra y en el conductor neutro. La importancia de esta transformación radica en que facilita la identificación rápida y precisa de anomalías y la realización de estudios de protección y calidad de servicio, ya que cada componente tiene un significado físico claro y permite descomponer fenómenos complejos en redes más sencillas de analizar ([Grainger and Stevenson, 1994](#)). En particular, el componente de secuencia cero se puede calcular directamente a partir de las corrientes medidas en cada fase y en el neutro, como $I_0 = (I_a + I_b + I_c)/3$, lo que evidencia la relevancia de monitorear la corriente de neutro para detectar fallas de alta impedancia o fugas a tierra que, de otro modo, podrían pasar desapercibidas ([Kundur, 1994](#)).

Asimismo, la medición individual de los voltajes de fase (V_a , V_b , V_c) y su relación con el neutro es esencial para evaluar la calidad del suministro eléctrico, dado que las cargas monofásicas, predominantes en redes de distribución urbana, generan constantemente desequili-

brios en tensiones y corrientes. Estos registros permiten aplicar también la transformación a componentes simétricas en voltajes (V_0, V_1, V_2), lo que ayuda a diagnosticar perturbaciones, fluctuaciones y armónicos en el sistema ([El-Sharkawi, 2008](#)). De esta manera, contar con datos precisos de I_a, I_b, I_c e I_n es imprescindible para la operación, mantenimiento y protección de subestaciones eléctricas, ya que a partir de estos datos se pueden detectar sobrecargas, fallas a tierra, desequilibrios de carga y otras anomalías que comprometen la estabilidad y seguridad del sistema eléctrico ([Anderson, 1995](#)). Por lo tanto, el análisis detallado de estas variables, complementado con el uso de componentes simétricas, constituye una herramienta técnica fundamental para garantizar la calidad, confiabilidad y seguridad en la distribución eléctrica trifásica.

1.2 Importancia del análisis de corrientes trifásicas

El análisis detallado de las corrientes por fase y en el conductor neutro en sistemas eléctricos trifásicos resulta esencial para asegurar una operación segura, eficiente y confiable de las redes de distribución. Este tipo de información permite identificar y diagnosticar una variedad de condiciones anómalas que pueden comprometer tanto el rendimiento del sistema como la calidad del suministro eléctrico. En condiciones ideales, un sistema trifásico balanceado presenta corrientes de igual magnitud en las tres fases (A, B y C), desfasadas 120 grados entre sí, mientras que la corriente en el neutro debería ser prácticamente nula. Cualquier desviación de este comportamiento puede ser indicativa de un desbalance en las cargas, fallos en equipos o irregularidades en la conexión a tierra ([Bergen and Vittal, 2000](#)).

La importancia de analizar estos datos radica en que desequilibrios entre fases o una corriente elevada en el neutro pueden provocar pérdidas por efecto Joule, sobrecalentamiento de los conductores, degradación prematura de transformadores y, en casos críticos, interrupciones del servicio eléctrico. Esto se vuelve especialmente relevante en sistemas donde predominan cargas monofásicas —como es frecuente en entornos residenciales, comerciales o incluso universitarios—, donde los desbalances son comunes y deben ser vigilados para preservar la vida útil de los componentes y mantener la eficiencia global del sistema ([Grainger and Stevenson, 1994](#)). A través de este tipo de monitoreo, es posible implementar acciones correctivas como el reequilibrio de cargas, la revisión del sistema de tierra o la incorporación de elementos de protección más adecuados.

Adicionalmente, el monitoreo de corrientes por fase y neutro permite detectar fallas a tierra o fugas eléctricas. Una corriente inusualmente elevada en el neutro puede ser un claro indicio de una falla de línea a tierra, la cual representa un riesgo tanto para la seguridad de las instalaciones como para la integridad del sistema eléctrico. Estas anomalías deben ser atendidas de manera inmediata ([Kundur, 1994](#)). El uso de herramientas como la teoría de componentes simétricos, que se basa precisamente en la medición de corrientes y voltajes por fase, posibilita un análisis más preciso del sistema, permitiendo detectar fallas asimétricas y mejorar la coordinación de los esquemas de protección ([Selic et al., 2010](#)).

En el contexto de redes complejas como la que alimenta a Ciudad Universitaria (CU), es habitual encontrar registros como I_a , I_b , I_c e I_n para corrientes, y V_a , V_b , V_c y V_n para voltajes. Esta estructuración responde a la naturaleza del sistema trifásico: cada fase transporta corriente de forma alterna y desfasada, lo que hace necesario analizar cada una de manera individual para evaluar el balance de carga, prevenir sobrecargas o identificar fallas puntuales. La corriente I_n , correspondiente al conductor neutro, es especialmente útil para detectar asimetrías entre las fases. Asimismo, se suelen registrar otros parámetros fundamentales como la frecuencia, las potencias activa, reactiva y aparente, e incluso la frecuencia total, con el fin de evaluar la calidad de la energía entregada.

Estas mediciones forman parte integral de sistemas de supervisión y adquisición de datos en tiempo real, como SCADA (Supervisory Control and Data Acquisition), que son esenciales para la gestión moderna de redes eléctricas. Gracias a este tipo de herramientas, es posible realizar estudios de calidad de energía, mantenimiento predictivo, análisis de eficiencia energética y planificación de la expansión del sistema. En entornos con múltiples subestaciones —como ocurre en CU, con más de 170 subestaciones derivadas y 7 generales interconectadas por rutas subterráneas de media tensión— este análisis se vuelve indispensable para garantizar un suministro eléctrico confiable y adaptado a las necesidades de una comunidad compleja y diversa.

Metodología

Los objetivos del proyecto son realizar un análisis estadístico de mediciones eléctricas reales obtenidas en CU y almacenadas en archivos CSV, considerando variables estadísticas, además de identificar patrones de distribución. También se propone efectuar un análisis detallado de un conjunto de datasets.

Asimismo, se plantea desarrollar un mecanismo automatizado que permita procesar de manera recurrente grandes volúmenes de archivos y generar los cálculos estadísticos correspondientes. Posteriormente, diseñar e implementar una base de datos para almacenar los resultados consolidados de los análisis y establecer un proceso que asegure su actualización continua. Finalmente, se considera crear una interfaz de visualización que permita explorar tendencias, realizar comparaciones y observar patrones de manera clara y accesible para distintos perfiles de usuarios.

2.1 Fundamentos de estadística

En el marco de este proyecto, se recurrió a herramientas del análisis de datos con el propósito de identificar patrones y analizar comportamientos en los datos. Estas herramientas consisten en la interpretación de medidas de tendencia central, como la media aritmética, la mediana y la moda. Estas medidas de tendencia central permiten resumir el comportamiento general de una variable numérica, ofreciendo un punto de referencia respecto al cual se pueden evaluar desviaciones o comportamientos atípicos.

Desde el enfoque estadístico, los datos utilizados pueden considerarse como una muestra extraída de una población más amplia. Este procedimiento, conocido como muestreo, es común en análisis de datos. El análisis se basa en inferencias extraídas a partir de esta muestra, combinando estadística descriptiva, que permite resumir y visualizar los datos, con elementos de estadística inferencial, que sirven para extender conclusiones hacia la población completa.

Dado que se analizaron los datos para cada corriente eléctrica (Ia, Ib, Ic, In), fue importante clasificar en el marco de la estadística, el tipo de variable utilizada para su correcto estudio. Estas variables fueron clasificadas como cuantitativas, variables que representan cantidades numéricas y permiten realizar operaciones matemáticas. A su vez, fueron clasificadas como continuas, ya que pueden tomar un número infinito de valores dentro de un intervalo.

Esta clasificación permitió construir gráficos estadísticos para representar visualmente los datos de manera informativa. Para variables cuantitativas continuas, se utilizó el histograma de frecuencia como herramienta principal. Este tipo de gráfica permite observar cómo se distribuyen las observaciones a lo largo de diferentes intervalos (o “clases”) de valores, mostrando la frecuencia proporcional con que los datos caen en cada intervalo. Su construcción implicó dividir el rango de valores en subintervalos de igual longitud, y posteriormente contar el número de datos que caían en cada uno. Un ejemplo de histograma de frecuencia se observa en la Figura 3

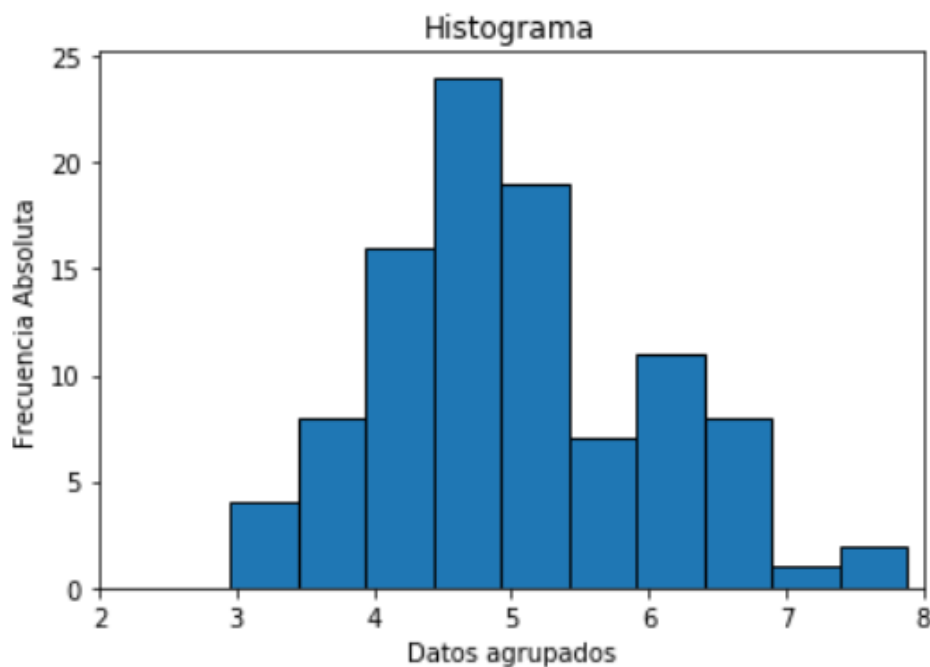


Figura 3: Representación de un histograma

Una vez construido el histograma, fue posible observar el comportamiento general de la distribución de la variable y ajustar una curva de probabilidad dependiendo de la forma que presentaba la gráfica. Para ello, se evaluó si la distribución era simétrica, sesgada, unimodal o multimodal. Un ejemplo de estas distribuciones se puede ver en la Figura 4. Este tipo de ajuste se aplicó a todas las variables contenidas en el archivo CSV, aunque el análisis detallado se centró únicamente en la corriente eléctrica. El objetivo fue identificar el tipo de distribución de cada variable para, posteriormente, automatizar el análisis realizado con la corriente y extenderlo a otras variables, como el voltaje, de acuerdo con su distribución estadística.

Un histograma simétrico (o unimodal) como el de la Figura 3, centra los datos en la media (medida central) y tiene características a través de la distribución de la media y la desviación estándar. Una característica del histograma simétrico es contener la mayor cantidad de datos

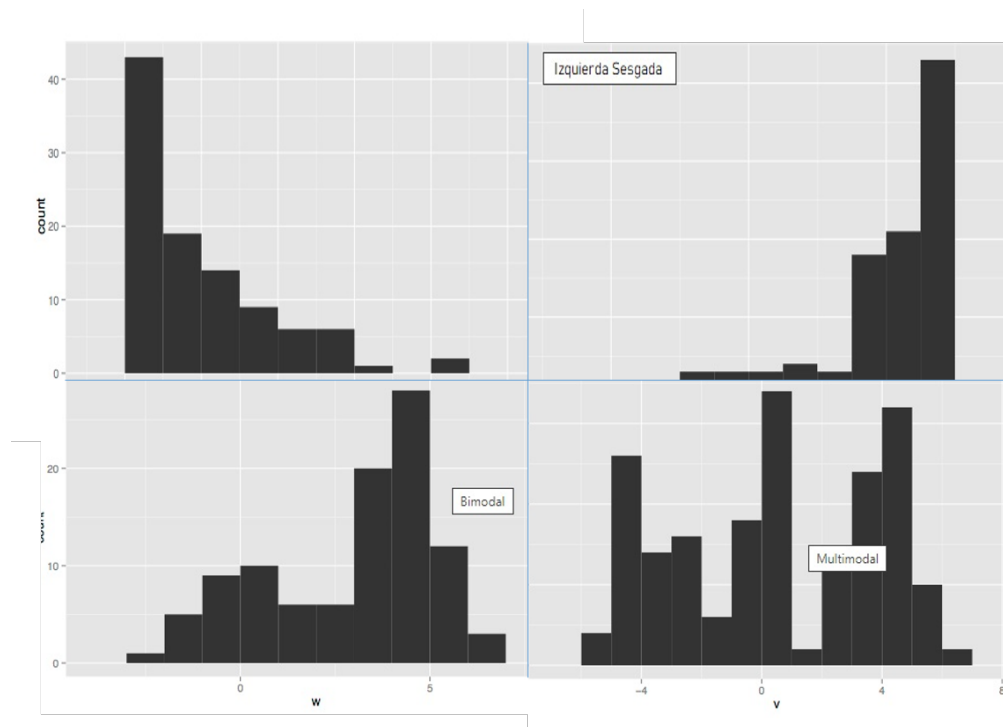


Figura 4: Histogramas que ilustran distintos tipos de distribuciones de datos. En el panel superior izquierdo se observa una distribución sesgada a la derecha (asimetría positiva), mientras que en el panel superior derecho se muestra una distribución sesgada a la izquierda (asimetría negativa). En el panel inferior izquierdo aparece una distribución bimodal, caracterizada por dos picos o modas. Finalmente, en el panel superior derecho se presenta una distribución multimodal, con múltiples picos, indicando varios grupos o patrones en los datos.

del centro del gráfico. En estadística, el ajuste a este modelo se llama normal y le permite analizar cuánto se desvían otros datos de este modelo. Un histograma está sesgado hacia la derecha cuando la distribución de datos indica que los valores altos ocurren con baja frecuencia. Este modelo también se conoce comúnmente como el modelo de “cola derecha”, ya que “se afina.” a medida que nos movemos a lo largo del eje x, lo que indica que la frecuencia está disminuyendo. Un ejemplo de este tipo de distribución se observa en el panel superior derecho de la Figura 4.

Se le llama histograma sesgado a la izquierda como el panel superior derecho de la Figura 4 cuando la frecuencia de los datos se concentra en los valores altos del lado izquierdo a medida que recorremos el eje x. Por tanto, también se le conoce como histograma con “cola a la izquierda”, por la misma razón que el anterior, ya que a la izquierda formamos una especie de cola debido a la baja frecuencia de los datos al principio. Se observa que hay más información por encima de la media por falta de simetría. Se llama histograma bimodal (ver panel inferior izquierdo de la Figura 4) cuando aparecen dos picos. Así sabemos que en dos momentos diferentes hay una concentración de frecuencia que sobresale. Un histograma es multimodal cuando aparecen varios picos (ver panel inferior derecho de la Figura 4). Los picos indicarán el mayor número de ocurrencias.

2.2 Medidas de tendencia central

Procedemos a definir de manera más específica la forma de resumir datos cuantitativos en índices estadísticos numéricos, que nos permitirán comparar con facilidad diferentes conjuntos de datos. La mediana y la media son dos ejemplos de ello. La media aritmética de un conjunto de datos cuantitativos es su valor medio (promedio). Para calcular la media aritmética, simplemente sumamos todos los valores y dividimos por el número total de datos n . Utilizando la notación habitual, la media aritmética es igual a:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (1)$$

donde $\bar{x}$ representa la media aritmética, n es el número total de observaciones, y x_i es el valor de cada dato en el conjunto. Esta medida es útil cuando los datos no presentan valores atípicos extremos, ya que dichos valores pueden sesgar la media y dar una idea equivocada del comportamiento central del conjunto de datos.

Por otro lado, la mediana es una medida de tendencia central que representa el valor que ocupa la posición central en un conjunto de datos ordenados. Es decir, es el valor que divide la distribución en dos partes iguales: el 50 % de las observaciones se encuentran por debajo de ella y el otro 50 % por encima. Para determinarla, es necesario ordenar los datos de menor a mayor y localizar el valor situado en la posición:

$$\text{Posición de la mediana} = \frac{n + 1}{2} \quad (2)$$

donde n es el número total de observaciones. Si n es impar, la mediana es el valor que se encuentra en dicha posición. Si n es par, se obtiene como el promedio de los dos valores centrales. Esta medida es especialmente útil en distribuciones sesgadas o con valores atípicos, ya que no se ve afectada por extremos como lo hace la media.

Tanto la mediana como la media aritmética miden el centro de la distribución, pero lo hacen de manera distinta. Sólo cuando la distribución es simétrica las dos medidas coinciden. La principal diferencia entre ambas es cómo se ven afectadas por las asimetrías o los datos alejados. Cuando la distribución es asimétrica, la media aritmética siempre se desplaza hacia la cola de la distribución. La presencia de un valor muy elevado no afecta a la mediana, pero influye mucho sobre la media aritmética. Decimos que la mediana “resiste” los datos alejados. Un ejemplo de esto se muestra en la Figura 5, donde la distribución está sesgada a la derecha: la moda es el valor más frecuente, la mediana se encuentra más a la derecha y la media aún más desplazada hacia la cola, reflejando la influencia de los valores extremos.

Para complementar el análisis de los datos, no es suficiente conocer únicamente las medidas de tendencia central, ya que estas indican dónde se encuentran los valores centrales de una distribución, pero no ofrecen información sobre cómo se dispersan o agrupan las observaciones alrededor de ese centro. Por ello, es necesario considerar también las medidas de dispersión, que cuantifican el grado de variabilidad presente en los datos. Una medida básica

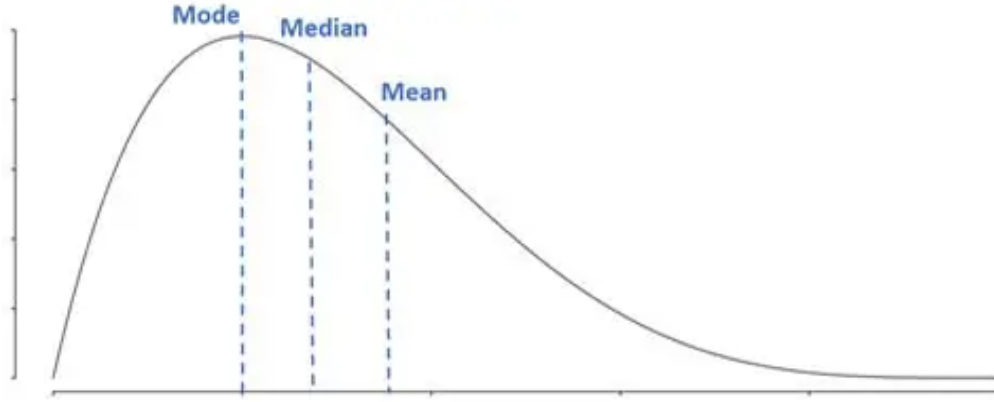


Figura 5: Distribución sesgada a la derecha (asimetría positiva), en la que se observa la posición relativa de la moda, la mediana y la media aritmética. La moda representa el valor más frecuente, la mediana divide a la distribución en dos partes iguales, y la media se ve desplazada hacia la cola debido a la influencia de valores extremos.

y sencilla es el rango, que se calcula como la diferencia entre el valor máximo $x_{(n)}$ y el valor mínimo $x_{(1)}$ del conjunto de datos, es decir,

$$R = x_{(n)} - x_{(1)} \quad (3)$$

sin embargo, su utilidad es limitada debido a que un solo valor extremo puede alterar significativamente esta medida. Para un análisis más completo, se emplea la **varianza**, definida como la media de los cuadrados de las diferencias entre cada dato x_i y la media aritmética $\bar{x}$, expresada por la fórmula

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2, \quad (4)$$

lo cual evita cancelaciones entre valores positivos y negativos y otorga mayor peso a las desviaciones mayores. Debido a que la varianza está expresada en las unidades al cuadrado de la variable, resulta más interpretativa su raíz cuadrada, conocida como desviación estándar, que se calcula como

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}, \quad (5)$$

y expresa la dispersión en las mismas unidades que los datos originales, siendo uno de los índices de dispersión más utilizados. Adicionalmente, los cuartiles son índices de posición que dividen el conjunto ordenado de datos en cuatro partes iguales: el primer cuartil Q_1 deja el 25 % de las observaciones por debajo, el segundo cuartil Q_2 corresponde a la mediana o 50 %, y el tercer cuartil Q_3 deja el 75 % de las observaciones por debajo. La diferencia entre el tercer y primer cuartil, llamada rango intercuartílico,

$$RI = Q_3 - Q_1, \quad (6)$$

mide la dispersión del 50 % central de los datos, proporcionando una medida robusta frente a valores atípicos. Finalmente, los percentiles generalizan esta división, dividiendo la distribución en 100 partes iguales, de modo que el percentil P_k indica el valor bajo el cual se encuentra el k % de las observaciones, formalmente expresado como

$$P_k = \text{valor tal que al menos } \frac{k}{100} \times n \text{ observaciones son menores o iguales a } P_k.$$

En conjunto, estas medidas de dispersión complementan a las medidas de tendencia central para proporcionar una descripción completa y precisa de las características de un conjunto de datos.

2.3 Outliers (Valores atípicos)

Se denominan valores atípicos u outliers a aquellas observaciones con características diferentes de las demás. Son elementos que pueden no ser representativos de la población, pudiendo distorsionar seriamente el comportamiento de los contrastes estadísticos. Por otra parte, aunque diferentes a la mayor parte de la muestra, pueden ser indicativos de las características de un segmento válido de la población y, por consiguiente, una señal de la falta de representatividad de la muestra.

Los casos atípicos pueden clasificarse en cuatro categorías principales: aquellos que se deben a errores de procedimiento, como fallos en la entrada o codificación de datos, los cuales deben corregirse o eliminarse; los que resultan de eventos extraordinarios y no representan a la población, por lo que pueden excluirse del análisis; las observaciones inusuales pero válidas dentro del rango de las variables, que deben conservarse analizando su influencia en los modelos; y finalmente, los casos sin explicación aparente, para los cuales se recomienda realizar el análisis con y sin ellos, reportar su impacto si resultan influyentes y tratar de comprender su origen.

Para resolver estas limitaciones, podemos utilizar la prueba de puntuación z-score. Este método se basa en los propios datos para determinar qué valores son inusuales en comparación con el resto. Es uno de los métodos más utilizados para este propósito debido a su flexibilidad y precisión. La puntuación Z mide cuántas desviaciones estándar está cada valor por encima o por debajo de la media del conjunto de datos. De esta manera, permite identificar valores que se desvían significativamente del promedio, sin necesidad de establecer umbrales arbitrarios o confiar en criterios externos. La puntuación Z-score se calcula como:

$$\text{z-score} = \frac{\text{valor} - \text{promedio}}{\text{desviación estándar}} \quad (7)$$

El método del z-score no es robusto porque los valores extremos (outliers) pueden influir considerablemente en los cálculos de la media y la desviación estándar, afectando la capacidad del método para identificar otros outliers. Ser robusto significa que un método es resistente a la influencia de valores atípicos y produce resultados confiables incluso en presencia de estos.

Un método preferido y también muy estándar para identificar valores extremos utiliza el rango intercuartílico (IQR, por sus siglas en inglés). La principal ventaja del IQR es que es mucho más robusto frente a los outliers que los z-scores. Al igual que los z-scores, el método del IQR encuentra valores que están alejados de la media, en relación con la variabilidad normal de los datos, aunque en lugar de usar la media para el cálculo del promedio, utiliza la mediana, y para la variabilidad usa el IQR en lugar de la desviación estándar. Para calcular el IQR, primero calculamos el primer y el tercer cuartil (es decir, los valores en los percentiles 25 y 75). Luego, restamos estos valores para obtener el IQR. Los umbrales para los outliers se definen como:

$$\text{Umbrales para outliers} = Q_1 - (2,2 \times IQR) \quad \text{y} \quad Q_3 + (2,2 \times IQR) \quad (8)$$

La desviación absoluta mediana (MAD, por sus siglas en inglés) es otra medida de dispersión, similar al IQR y más robusta, nuevamente, a los valores atípicos. Para obtener la MAD para un conjunto de datos, primero calculamos la mediana de los datos. Luego, para cada punto, tomamos la diferencia absoluta con respecto a la mediana, y luego calculamos la mediana de estas diferencias. Para determinar si un punto es un valor atípico usando la MAD, calculamos qué tan lejos está el punto de la mediana (usando la diferencia absoluta). Luego, dividimos esta diferencia entre la MAD para determinar qué tan lejos está de la mediana en relación con los otros puntos en los datos. Para determinar si un punto es un valor atípico, verificamos si el puntaje MAD supera un umbral preestablecido.

2.4 Procesamiento de datos

Los archivos CSV analizados fueron extraídos del sistema NAS, una base de datos del Instituto de Ingeniería de la UNAM. Cada archivo contiene columnas correspondientes a las corrientes eléctricas Ia, Ib, Ic, In; los voltajes Va, Vb, Vc, Vn; así como la frecuencia y la potencia. Antes de analizar en detalle el comportamiento de las corrientes, se construyeron histogramas para cada variable con el fin de identificar su distribución y ajustar un modelo estadístico acorde a la forma de los datos.

Para llevar a cabo el análisis automatizado de las señales de corriente por hora y detectar sus outliers, se desarrolló un script en Python que aprovecha principalmente la biblioteca Pandas para manejo de datos y Matplotlib para visualización. En primer lugar, los datos de entrada (en formato CSV) se cargaron en un DataFrame de Pandas usando funciones como `pd.read_csv()`. Tras la lectura, se estableció un índice por hora convirtiendo una columna de timestamps con `pd.to_datetime()`, de manera que cada registro estuviera asociado a su marca temporal correspondiente. Esto permitió ordenar y agrupar los datos según la hora de muestreo. En total, se dispuso de 17 280 muestras, equivalentes a 24 horas de registro a razón de 720 muestras por hora. Un diagrama de flujo en el apéndice muestra el proceso completo de análisis de corrientes eléctricas y detección de valores atípicos, desde la carga y preprocesamiento de datos hasta la limpieza y exportación de los resultados.

2.5 Análisis Exploratorio

Para identificar automáticamente las columnas que correspondían a mediciones de corriente, se filtró el DataFrame según los nombres de columna. Para ello, se empleó una comprensión de listas para seleccionar todas las columnas que contienen cierto prefijo o una palabra clave asociada a la variable de estudio. Este filtrado por etiqueta permitió obtener un subconjunto del DataFrame con solo las columnas de interés (cada una representa una variable de corriente distinta).

Una vez cargados y aislados los datos de corriente, el análisis por hora se efectuó fragmentando el conjunto de datos en bloques de 720 filas consecutivas, cada uno correspondiente a un intervalo de una hora. La indexación posicional de Pandas (`.iloc`) facilita esta tarea de muestreo. Por ejemplo, usando `df.iloc[i*720:(i+1)*720]` se extraen de forma iterativa 720 registros por hora. Esto equivale a dividir los 17 280 registros en 24 segmentos de igual longitud. Cabe destacar que `.iloc` acepta rangos tipo slice (como `:720`, `720:1440`, etc.) para seleccionar subconjuntos de filas enteras. De esta forma, en un bucle o función de agrupamiento se recorrieron los datos hora por hora.

Para cada bloque horario y cada columna de corriente se calcularon las estadísticas descriptivas necesarias. En concreto, se emplearon los métodos de Pandas para obtener la media, la desviación estándar y la varianza de la muestra. Pandas proporciona `DataFrame.mean()`, `DataFrame.std()` y `DataFrame.var()` para este propósito. Por defecto, `mean()` y `std()` devuelven la media y la desviación estándar (muestral) de cada columna, ignorando valores faltantes (NA). La varianza se obtiene con `var()`, que por defecto usa $N-1$ como divisor (varianza sin sesgo). En el código, se aplicó, por ejemplo, `media = bloque[columna].mean()`, `desv = bloque[columna].std()`, `var = bloque[columna].var()`, almacenando estos resultados para cada hora y cada variable.

Con las estadísticas calculadas por hora, se procedió a la detección de outliers usando el criterio del percentil. Se definieron umbrales en los percentiles extremos: 0.1 % (cuantil 0.001) y 99.9 % (cuantil 0.999) debido a que fueron los valores que mejor se ajustaban a la distribución de los datos. Para obtener estos umbrales se empleó el método `DataFrame.quantile(q)`, donde q es el valor del cuantil buscado. Por ejemplo, `límite_inf = bloque[columna].quantile(0.001)` y `límite_sup = bloque[columna].quantile(0.999)`. La documentación de Pandas indica que `quantile(q)` devuelve el valor en el cuantil q pedido (con $0 \leq q \leq 1$). Alternativamente, podría usarse `numpy.percentile`, pero se mantuvo el enfoque con Pandas. Con los valores de percentil, se marcaron como outliers todos los registros cuya medición de corriente estuviera por debajo del umbral inferior o por encima del umbral superior de cada hora y cada variable.

Cada outlier detectado se registró en una estructura conjunta para su posterior análisis. Se fue almacenando para cada valor anómalo su hora (el índice de bloque horario), el nombre de la variable de corriente y el valor observado. Al terminar la inspección de todos los datos, se creó un DataFrame (por ejemplo, usando `pd.DataFrame(lista_outliers)`) que contiene una fila por cada outlier, con columnas como “Hora”, “Variable” y “Valor”. Este marco de datos permitió compilar un reporte completo de todos los outliers detectados en el conjunto de

muestras.

Con la lista de outliers identificados, el paso siguiente fue limpiar la señal original reemplazando esos valores por “faltantes” (NaN). Para ello se generaron condiciones lógicas que indicaban la posición de cada outlier, y se asignó `np.nan` a dichas posiciones. Una forma eficiente en Pandas es usar `DataFrame.mask(condición)`, que reemplaza con NaN todos los valores que cumplen la condición dada. Por ejemplo: `df_hora = df_hora.mask(df_hora[columna] < límite_inf)` y de forma similar para el límite superior. De este modo, las filas de datos originales conservan los valores normales, pero los outliers quedan convertidos en valores NaN. Este procedimiento evita remover filas completas y mantiene alineados los registros temporales, a la vez que hace explícita la ausencia de dato en los puntos extremos.

Tras eliminar los valores extremos, la señal horario-por-horario contenía huecos (NaN). Para recomponer la serie de forma continua se aplicó interpolación lineal. Pandas cuenta con el método `interpolate(method='linear')` para rellenar valores faltantes linealmente. En el código se utilizó `df.interpolate(method='linear', inplace=True)`, lo que interpola en cada columna asumiendo que los índices (filas) están equidistantes en el tiempo. De esta forma, cada valor NaN se reemplaza por la línea recta entre sus vecinos válidos inmediatos. Esta técnica rellena los huecos producidos por la eliminación de outliers sin introducir saltos bruscos y preservando la tendencia local de la señal.

2.6 Visualización

Para verificar visualmente el resultado y comunicar los límites usados, se graficó la serie limpia junto con líneas que muestran las estadísticas calculadas. Usando Matplotlib se trazó la curva de la señal interpolada en cada bloque horario (por ejemplo con `plt.plot()`). Adicionalmente, se dibujaron líneas horizontales que marcan la media y las bandas de desviación estándar, así como los valores de percentil calculados. La función `matplotlib.pyplot.axhline(y=valor, ...)` permite añadir una línea horizontal en la posición y deseada. Por ejemplo, se dibujó `axhline` en `y=media` (media por hora), en `y=media+desv` y `y=media-desv` para ilustrar $\pm 1\sigma$, y en `y=límite_inf` y `y=límite_sup` para visualizar los percentiles 0.1 % y 99.9 %. Cada línea se configuró con colores y estilos diferenciados (continua, punteada, etc.) y leyenda adecuada. Este gráfico permite apreciar en cada hora el comportamiento de la señal con respecto a sus estadísticas, facilitando la validación del proceso.

Para mejorar la exploración de la gráfica, se agregó interactividad con la biblioteca `mplcursors`. Esta extensión permite que al mover el cursor sobre la figura se muestren anotaciones o tooltips con información de los datos subyacentes. En el script se importó `mplcursors` y se invocó `mplcursors.cursor(hover=True)`, lo cual activa un cursor interactivo sobre los objetos dibujados (por ejemplo los puntos o líneas). De este modo, al pasar el ratón por encima de la curva o líneas horizontales, aparece automáticamente una etiqueta con valores numéricos (p.ej. coordenadas x-y). La documentación de `mplcursors` indica que esta herramienta facilita la selección y lectura de puntos en plots de Matplotlib. Gracias a ello, el usuario puede ins-

peccionar individualmente los valores de corriente y su contexto sin necesidad de impresión de valores adicionales por código.

Por último, los valores de outliers recopilados se exportaron a un informe en Excel para registro externo. Pandas simplifica esta tarea con `DataFrame.to_excel()`. Con un comando del tipo `df_outliers.to_excel("outliers_detectados.xlsx", sheet_name="Outliers", index=False)`, se escribe el `DataFrame` de outliers en un archivo `.xlsx`. La documentación de señala que basta indicar el nombre de archivo como destino para generar la hoja de cálculo. De este modo se obtuvo un archivo de Excel estructurado, en el que cada fila corresponde a un outlier detectado, con sus columnas de hora, variable y valor. Este reporte externo puede ser revisado o enviado como resultado de la metodología.

Resultados

Se graficaron inicialmente los histogramas de los datos contenidos en el archivo CSV, enfocándose particularmente en las corrientes eléctricas (I_a , I_b , I_c , I_n). Estos histogramas, mostrados en la Figura 6, revelan una distribución asimétrica con sesgo hacia la derecha. Este tipo de distribución indica que los valores altos, aunque poco frecuentes, elevan el promedio (media), haciendo que esta medida central no coincida necesariamente con el valor más representativo (la moda o la mediana) de los datos. El código que muestra estos histogramas se puede visualizar en el apéndice .

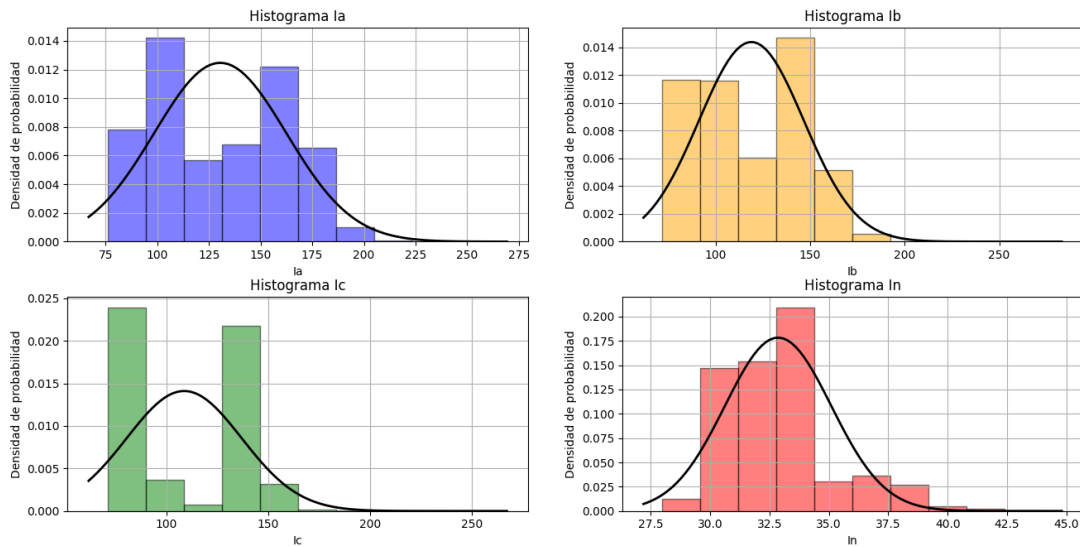


Figura 6: Histogramas de las corrientes I_a , I_b , I_c e I_n , empleados para visualizar la distribución de los datos, los cuales muestran una asimetría con sesgo positivo.

Una vez identificado el tipo de distribución de los datos, se procedió a comparar distintos métodos para la detección de valores atípicos. Tal como se mencionó anteriormente, se utilizaron tres enfoques: el método del rango intercuartílico (IQR), el método de la desviación absoluta mediana (MAD) y el método basado en percentiles. En la Figura 7 se presenta un histograma correspondiente a la corriente I_a , donde se ha superpuesto tres pares de líneas verticales que indican los límites definidos por cada método: el primer par corresponde a los umbrales del IQR, el segundo al MAD y el tercero a los percentiles acumulados en los extremos de la distribución.

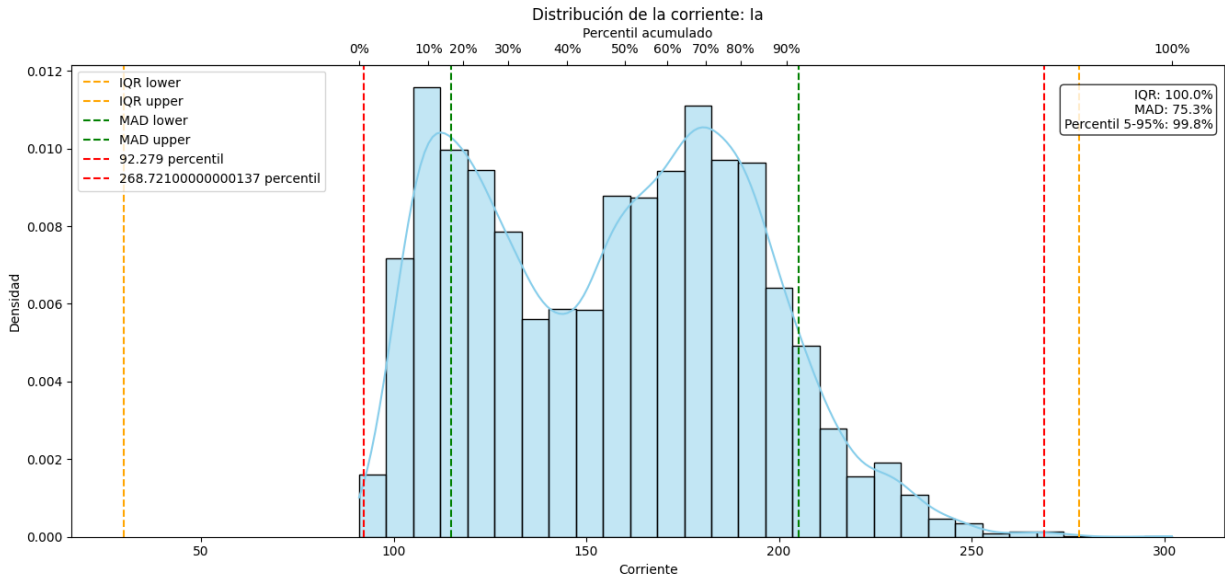


Figura 7: El histograma y la curva de densidad muestran la variabilidad de la corriente Ia, destacando límites para detectar valores atípicos según el rango intercuartílico, la desviación absoluta mediana y los percentiles 1 % y 99 %.

Tras el análisis visual y cuantitativo, se concluyó que el método de los percentiles resulta el más adecuado para este caso. A diferencia de otros métodos que se basan en la variabilidad estadística de los datos, el enfoque por percentiles establece límites en función de la posición relativa de los valores dentro de la distribución, lo que permite una detección más directa y flexible de los extremos. Esta característica es especialmente útil en distribuciones que presentan colas pesadas o asimetrías, ya que evita que los valores extremos sesguen el cálculo de parámetros como la desviación estándar o la mediana. No obstante, es importante tener cuidado en la elección de los percentiles, ya que si se eligen umbrales demasiado estrechos se corre el riesgo de excluir datos válidos. En este estudio, se seleccionaron percentiles que equilibran la sensibilidad a los valores atípicos sin comprometer la validez de los datos representativos. Estos percentiles tomaron el valor del 1 % y el 99 %, valores que se pueden visualizar en el código del apéndice .

Posteriormente, se llevó a cabo el análisis exploratorio del conjunto de datos utilizando la metodología descrita en el Capítulo 3. Para ello, se graficaron las corrientes correspondientes a un período de 24 horas del dataset. El código implementado para esta sección se presenta en el Apéndice . Algunas gráficas que dieron como resultado la implementación de este código se muestran en las figuras 8 y 9.

Dichas gráficas ilustran el comportamiento horario de las corrientes, y en ellas se aplicaron las estadísticas mencionadas previamente en el Capítulo 3. Específicamente, se incluyeron líneas punteadas que representan la media, los percentiles empleados para detectar valores atípicos, así como el umbral definido por el método del rango intercuartílico (IQR). Cabe destacar la presencia de valores notablemente altos que sobrepasan el límite superior del

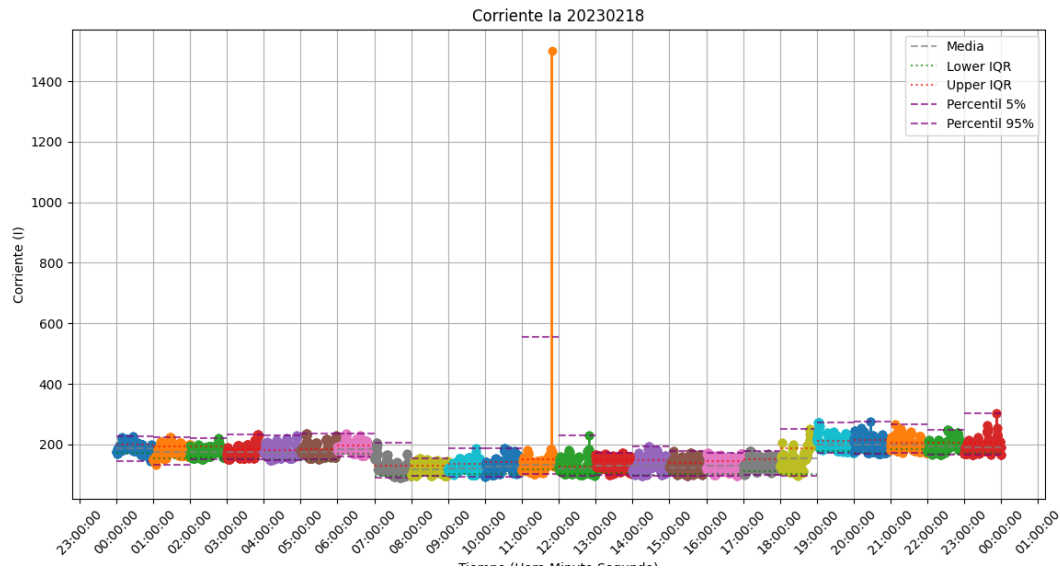


Figura 8: Gráfica del comportamiento de la corriente Ia ante la presencia de un valor atípico.

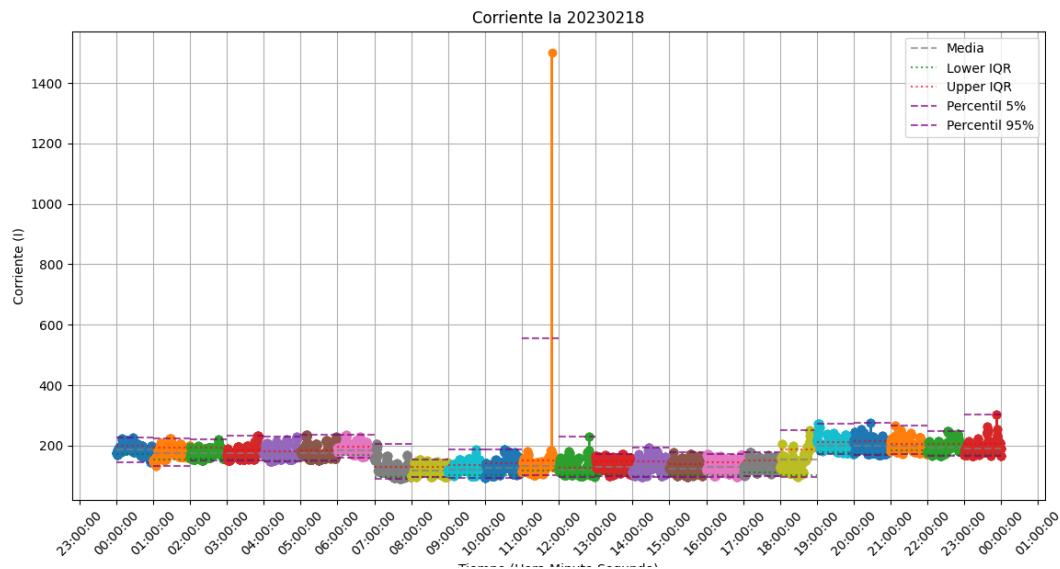


Figura 9: Gráfica del comportamiento de la corriente Ib ante la presencia de un valor atípico

percentil, lo cual permite identificarlos como outliers de acuerdo con los criterios establecidos.

A continuación, se procedió a detectar los valores atípicos utilizando la misma metodología descrita en el Capítulo 3, la cual puede consultarse en el código incluido en el Apéndice. Una vez implementado el algoritmo, se muestran dos resultados en las figuras 10 y 11. En dichas figuras se observa un comportamiento más estable respecto a las figuras 8 y 9, evidenciando la eliminación de los valores atípicos previamente detectados. Posteriormente, se aplicaron

las métricas estadísticas de interés a estos datos depurados con el objetivo de identificar posibles patrones en el comportamiento de las corrientes. Según la simbología mostrada en la gráfica, se incluyen la media, la dispersión representada por $\text{media} \pm \sigma$, así como los valores percentilares establecidos.

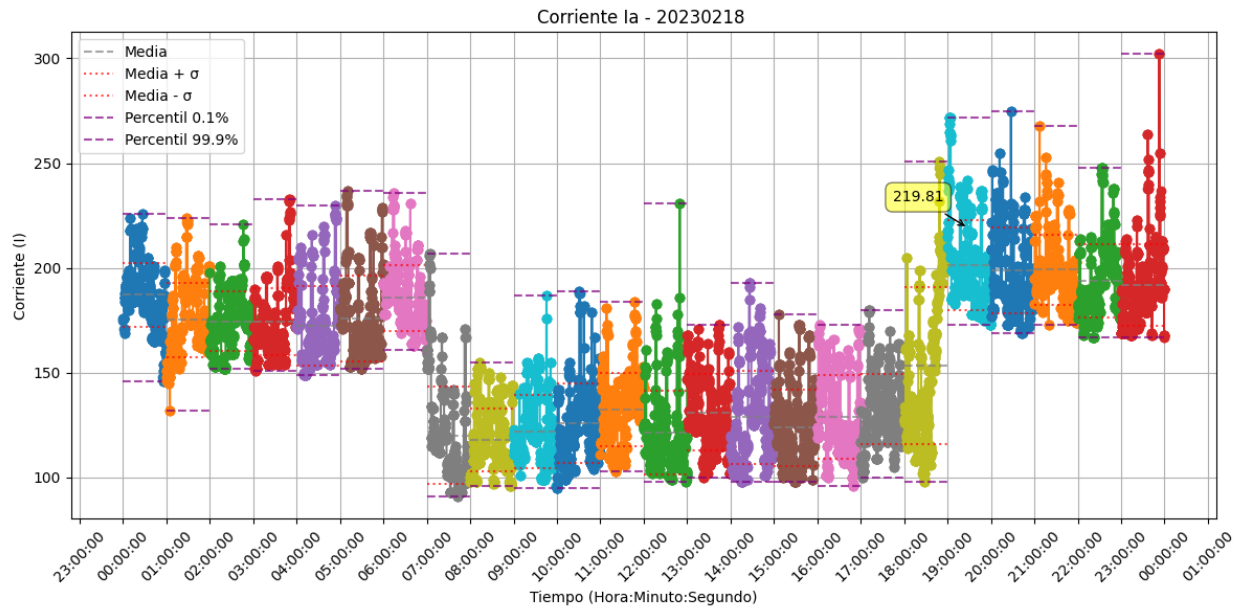


Figura 10: Gráfica del comportamiento de la corriente Ia sin valores atípicos.

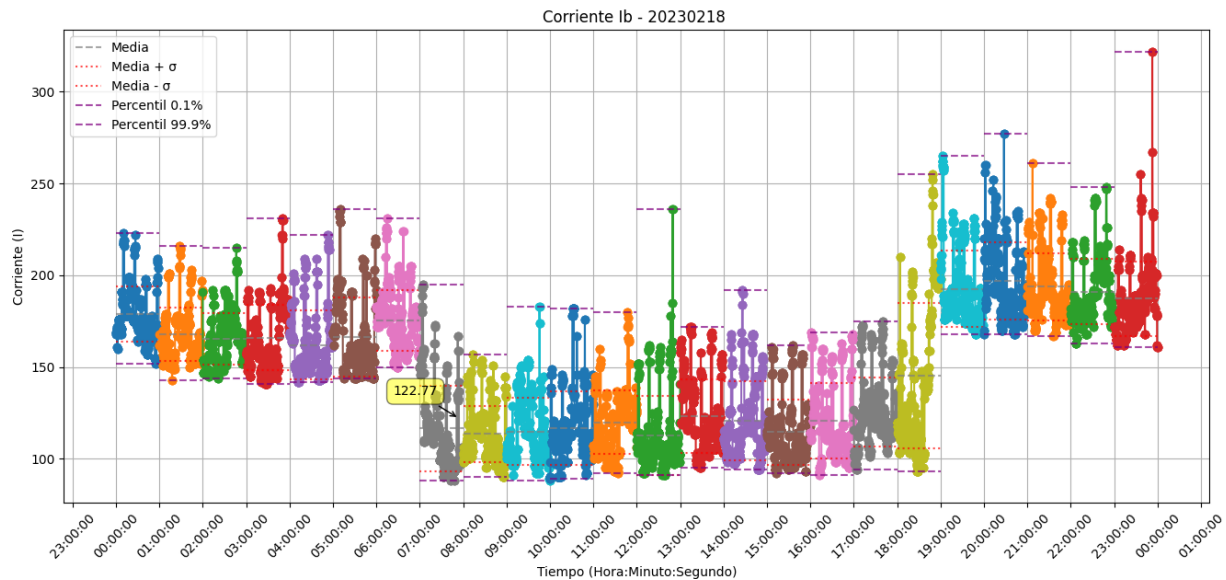


Figura 11: Gráfica del comportamiento de la corriente Ib sin valores atípicos

El comportamiento de ambas corrientes, Ia e Ib (ver figuras 10 y 11), muestran un patrón

similar. Durante las primeras horas del día, desde medianoche hasta aproximadamente las 06:00 am, la corriente se mantiene en un rango relativamente alto, con valores que oscilan principalmente entre 150 y 220 amperios. Posteriormente, entre las 06:00 am y las 18:00 pm, se observa una disminución notable en la corriente, que baja a un rango aproximado de 90 a 160 amperios. Finalmente, a partir de las 18:00 horas, la corriente vuelve a incrementarse, alcanzando picos de casi 300 amperios justo antes de la medianoche. En cuanto a la dispersión de los datos, esta es mayor en los periodos donde la corriente es alta, especialmente durante la noche y madrugada, mientras que durante el día los valores se concentran más cerca de la media, indicando menor variabilidad. Se detectan también picos destacados en ambas corrientes alrededor de las 23:00 horas, con valores cercanos a los 300 amperios, los cuales superan ampliamente el percentil 99.9 %, lo que indica que son eventos poco frecuentes o transitorios.

Es importante señalar que este análisis corresponde al comportamiento de la corriente eléctrica en la Torre de Rectoría de la UNAM, cuyo patrón se alinea con el ciclo de consumo característico de edificios de oficinas o instalaciones comerciales. Durante el horario laboral habitual (alrededor de 08:00 a 18:00 horas), el consumo eléctrico es más bajo o estable, debido a un uso moderado de equipos eléctricos y sistemas de iluminación. En contraste, fuera del horario laboral, especialmente en la madrugada y noche, la corriente aumenta, probablemente por la operación continua de sistemas automáticos como aire acondicionado, sistemas de seguridad, servidores, o equipos que se activan en horarios nocturnos para mantenimiento o limpieza. Los picos observados justo antes de la medianoche pueden estar relacionados con el arranque de motores o equipos que funcionan en ciclos, como bombas o sistemas de climatización, así como con procesos programados o fluctuaciones en la red eléctrica. La disminución de la corriente durante el horario laboral, aunque pueda parecer contraintuitiva, se explica si la carga principal corresponde a equipos que operan fuera de horario de oficina o si durante el día hay menor demanda de ciertos sistemas automáticos.

Finalmente, los valores atípicos fueron recopilados y organizados en una hoja de Excel. En la tabla 1 se presentan los valores atípicos identificados para cada corriente.

Tiempo	Ia (A)	Ib (A)	Ic (A)	In (A)
11:48:45	1501	165	141	41
10:58:00	118	2000	109	43
16:31:20	118	111	980	39
7:25:45	119	116	84	1050

Tabla 1: Valores atípicos de corriente (Ia, Ib, Ic e In) medidos en distintos instantes de tiempo durante la stampa de medición.

Para profundizar en el análisis, sería útil contar con datos adicionales de consumo real de potencia eléctrica y conocer con precisión qué equipos están activos en cada horario. También es importante verificar la calidad de la señal para descartar que algunos picos puedan deberse a ruido o errores en la medición. Finalmente, comparar estos datos con registros de

días normales y fines de semana permitiría confirmar si el patrón de comportamiento es consistente a lo largo del tiempo.

Conclusión

A partir del análisis de datos realizados, las corrientes muestran un patrón con dos niveles principales: valores más altos durante la noche y madrugada, y valores más bajos durante el horario laboral. Los picos nocturnos son eventos poco frecuentes pero esperables debido a equipos que funcionan en ciclos. Este comportamiento es normal para un edificio de oficinas con sistemas automatizados. No obstante, es recomendable realizar análisis complementarios para identificar con mayor certeza las causas específicas de estas variaciones.

Si bien estos análisis complementarios formaban parte de los objetivos planteados en el programa del servicio social, el tiempo disponible no fue suficiente para llevarlos a cabo. No obstante, el preámbulo ya está establecido, pues los códigos desarrollados fueron diseñados para que, al introducir cualquier archivo correspondiente a distintos días del año, pueda ejecutarse el mismo análisis descrito en los capítulos anteriores, permitiendo así realizar comparativas más completas.

Este servicio social me permitió aplicar conocimientos de física, estadística y programación en un contexto interdisciplinario con impacto social. Se fortalecieron tanto habilidades técnicas como de comunicación, y se contribuyó al entendimiento del panorama energético de Ciudad Universitaria.

Bibliografía

P. M. Anderson, *Power System Protection*. IEEE Press, 1995.

A. R. Bergen y V. Vittal, *Power Systems Analysis*. Prentice Hall, 2000.

Mohamed A. El-Sharkawi, *Electric Energy: An Introduction*. CRC Press, 2008.

J. J. Grainger y W. D. Stevenson, *Power System Analysis*. McGraw-Hill, 1994.

Prabha Kundur, *Power System Stability and Control*. McGraw-Hill, 1994.

B. Selic y colaboradores, *Symmetrical Components and Their Applications in Power Systems*, IEEE Tutorial, 2010.

Secretaría de Energía (SENER). (2012). *NOM-001-SEDE-2012, Instalaciones eléctricas (utilización)*. Diario Oficial de la Federación, México.

Short, T. A. (2014). *Electric Power Distribution Handbook* (2nd ed.). CRC Press.

Apéndice

Diagrama de flujo: Análisis de corrientes y outliers

El diagrama de flujo muestra de manera ordenada el proceso de análisis de corrientes eléctricas y detección de outliers, desde la carga y preprocesamiento de datos, pasando por la visualización y análisis de distribuciones, hasta la limpieza y exportación de los datos. A continuación, se presentan los códigos utilizados en el proyecto.

1. Carga y preprocesamiento de datos

- Importar CSV
- Seleccionar filas relevantes (ej. primeras 17,280)
- Eliminar duplicados y NaN
- Convertir columna t a datetime
- Seleccionar columnas de corrientes (Aquella que inicien con la letra I)

2. Visualización de series de tiempo

- Graficar serie de tiempo por columna
- Líneas de referencia: media, percentiles, IQR
- Formato eje X horas/minutos
- Interactividad con mplcursors

3. Histogramas y análisis de distribución

- Por cada hora (bloques de 720 muestras)
- Calcular histogramas de frecuencia
- Graficar distribuciones con percentiles y medianas

4. Gráficas con valores atípicos

- Calcular media, mediana, MAD, percentiles extremos
- Graficar series con líneas de referencia
- Detectar y marcar outliers

5. Detección y limpieza de outliers

- Detectar outliers con percentiles extremos
- Guardar outliers en outliers_global
- Reemplazar outliers por NaN e interpolar linealmente
- Guardar datos limpios en datos_limpios
- Calcular estadísticas (media, std, varianza, percentiles)
- Graficar datos limpios con líneas de referencia
- Exportar outliers_detectados.xlsx

6. Fin del análisis

Código comparación de métodos

A continuación se muestra el código en Python utilizado para la detección de outliers en corrientes eléctricas.

```
1 import numpy as np
2 import pandas as pd
3 import seaborn as sns
4 import matplotlib.pyplot as plt
5 from scipy.stats import median_abs_deviation
6
7 # Leer los datos
8 datos = pd.read_csv("C:\\Users\\Yahir\\Documents\\Documentos SS\\1SD
    -A1-2_MED-T1_20230218.csv")
9
10 datos_select = datos.iloc[:17280].dropna().drop_duplicates()
11 datos_select['t'] = pd.to_datetime(datos_select['t'])
12
13 columnas_corriente = [col for col in datos_select.columns if col.
    startswith('I')]
14
15 for column in columnas_corriente:
16     print(f"\n=== Analizando la corriente: {column} ===")
17
18     corriente = datos_select[column].dropna()
19
20     # --- Cálculos de límites ---
21
22     # IQR
23     q1 = corriente.quantile(0.25)
24     q3 = corriente.quantile(0.75)
25     iqr = q3 - q1
26     iqr_lower = q1 - 1.5 * iqr
27     iqr_upper = q3 + 1.5 * iqr
28
```

```
29 # MAD
30 mad = median_abs_deviation(corriente)
31 mad_lower = corriente.median() - 1.5 * mad
32 mad_upper = corriente.median() + 1.5 * mad
33
34 # Percentiles
35 percentile_5 = corriente.quantile(0.001)
36 percentile_95 = corriente.quantile(0.999)
37
38 # Outliers por método
39 outliers_iqr = (corriente < iqr_lower) | (corriente > iqr_upper)
40 outliers_mad = (corriente < mad_lower) | (corriente > mad_upper)
41 outliers_percentile = (corriente < percentile_5) | (corriente >
percentile_95)
42
43 total = len(corriente)
44 dentro_iqr = 100 * (~outliers_iqr).sum() / total
45 dentro_mad = 100 * (~outliers_mad).sum() / total
46 dentro_percentil = 100 * (~outliers_percentile).sum() / total
47
48 print("\n--- Porcentaje de datos dentro de cada método ---")
49 print(f"IQR: {dentro_iqr:.1f}%")
50 print(f"MAD: {dentro_mad:.1f}%")
51 print(f"Percentil 5%-95%: {dentro_percentil:.1f}%")
52
53 # --- Gráfico ---
54 plt.figure(figsize=(12, 6))
55 sns.histplot(corriente, bins=30, kde=True, color='skyblue', stat
='density')
56
57 plt.axvline(iqr_lower, color='orange', linestyle='--', label='
IQR lower')
58 plt.axvline(iqr_upper, color='orange', linestyle='--', label='
IQR upper')
59 plt.axvline(mad_lower, color='green', linestyle='--', label='MAD
lower')
60 plt.axvline(mad_upper, color='green', linestyle='--', label='MAD
upper')
61 plt.axvline(percentile_5, color='red', linestyle='--', label=f"{
percentile_5} percentil")
62 plt.axvline(percentile_95, color='red', linestyle='--', label=f"
{percentile_95} percentil")
63
64 plt.legend()
65 plt.title(f'Distribución de la corriente: {column}')
66 plt.xlabel('Corriente')
67 plt.ylabel('Densidad')
68
```

```

69     ax = plt.gca()
70     percentiles = np.linspace(0, 1, 11)
71     valores_percentiles = np.quantile(corriente, percentiles)
72     secax = ax.secondary_xaxis('top')
73     secax.set_xticks(valores_percentiles)
74     secax.set_xticklabels([f'{int(p*100)}%' for p in percentiles])
75     secax.set_xlabel('Percentil acumulado')
76
77     textstr = '\n'.join((
78         f'IQR: {dentro_iqr:.1f}%',
79         f'MAD: {dentro_mad:.1f}%',
80         f'Percentil 5-95%: {dentro_percentil:.1f}%'
81     ))
82
83     props = dict(boxstyle='round', facecolor='white', alpha=0.8)
84     plt.text(0.99, 0.95, textstr, transform=ax.transAxes,
85             fontsize=10, verticalalignment='top',
86             horizontalalignment='right', bbox=props)
87
88     plt.tight_layout()
89     plt.show()

```

Construcción de histogramas

```

1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from scipy.stats import norm
5  from sklearn.neighbors import KernelDensity
6
7  df = pd.read_csv('C:\\Users\\Yahir\\Documents\\Documentos SS\\1SD-A1
8  -2_MED-T1_20230702.csv')
9  df = df.dropna(subset = ['Van', 'Vbn', 'Vcn'], axis = 'index') #Con
10     esta función nos aseguramos de que no haya algún valor NAN en la
11     columna
12     #que queremos
13     graficar. Evitamos problemas al momento de graficar y hacer estad
14     ística.
15
16     #Creamos los gráficos
17     fig, axs = plt.subplots(1, 3, figsize=(18, 6)) #Las entradas 1 y 3
18     nos dicen que se trata de 1 fila, tres columnas
19
20     ax1 = axs[0] #establecemos que el primer histograma tome la primer
21     posición de nuestro arreglo gráfico

```

```
16
17 ax1 = df['Van'].hist(density=True, alpha=0.5, color='blue',
    edgcolor='black', ax=ax1) #Pandas tiene una función para
    elaborar el histograma
18
    #lo que hace density=True es normalizar el
    histograma para que el área total sea 1, lo que permite comparar
    con la curva de densidad.
19
    # alpha ajusta la transparencia del histograma
20 # Calcular los parámetros de la distribución normal (media y
    desviación estándar)
21 mu_van, std_van = norm.fit(df['Van']) #Con esto, lo que hacemos es
    ajustar una distribución normal a los datos dados. Recordemos que
    una distribución normal contiene
22
    # la media y la desviación estandar.
23 # Crear una serie de valores para la curva de la distribución
24 xmin_van, xmax_van = ax1.get_xlim() #lo que hace esta función es
    obtener los límites del eje X del histograma.
25 x_van = np.linspace(xmin_van, xmax_van, 100) # Usamos linspace para
    crear un intervalo entre xmin y xmax separados por 100 puntos
26 p_van = norm.pdf(x_van, mu_van, std_van) # Esto calcula la función
    de densidad de probabilidad de la distribución normal para cada
    punto en x usando los parámetros ajustados.
27
28 # Graficamos la curva de distribución
29 ax1.plot(x_van, p_van, 'k', linewidth=2) # k nos da el color de la
    curva y linewidth el ancho de la curva
30 ax1.set_xlabel('Van')
31 ax1.set_ylabel('Densidad de probabilidad')
32 ax1.set_title('Histograma Van')
33
34 # Histograma de 'Vbn'
35
36 ax2 = axs[1] #establecemos que el segundo histograma tome la segunda
    posición de nuestro arreglo gráfico
37 ax2 = df['Vbn'].hist(density=True, alpha=0.5, color='orange',
    edgcolor='black', ax=ax2)
38 mu_vbn, std_vbn = norm.fit(df['Vbn'])
39 xmin_vbn, xmax_vbn = ax2.get_xlim()
40 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100)
41 p_vbn = norm.pdf(x_vbn, mu_vbn, std_vbn)
42 ax2.plot(x_vbn, p_vbn, 'k', linewidth=2)
43 ax2.set_xlabel('Vbn')
44 ax2.set_ylabel('Densidad de probabilidad')
45 ax2.set_title('Histograma Vbn')
46
47 # Histograma de 'Vcn'
```

```
48
49 ax3 = axs[2] #establecemos que el tercer histograma tome la tercera
    posición de nuestro arreglo gráfico
50 ax3 = df['Vcn'].hist(density=True, alpha=0.5, color='green',
    edgecolor='black', ax=ax3)
51 mu_vcn, std_vcn = norm.fit(df['Vcn'])
52 xmin_vcn, xmax_vcn = ax3.get_xlim()
53 x_vcn = np.linspace(xmin_vcn, xmax_vcn, 100)
54 p_vcn = norm.pdf(x_vcn, mu_vcn, std_vcn)
55 ax3.plot(x_vcn, p_vcn, 'k', linewidth=2)
56 ax3.set_xlabel('Vcn')
57 ax3.set_ylabel('Densidad de probabilidad')
58 ax3.set_title('Vcn')
59
60 # Mostrar las gráficas
61 plt.tight_layout() # Ajusta el espaciado entre subgráficas
62 plt.show()
63
64 #Duda, como hacemos para saber el número máximo de bins? Ejemplo.
    bins=30
65
66 #En este caso no creo que sea un histograma, creo que se asemeja más
    a un gráfico de barras.
67
68 #Histograma de las corrientes
69 #Creamos los gráficos
70 fig, axs = plt.subplots(2, 2, figsize=(10, 8)) #Las entradas 1 y 3
    nos dicen que se trata de 1 fila, tres columnas
71
72 ax1 = axs[0,0] #establecemos que el primer histograma tome la primer
    posición de nuestro arreglo gráfico
73
74
75 ax1 = df['Ia'].hist(density=True, alpha=0.5, color='blue', edgecolor
    ='black', ax=ax1) #Pandas tiene una función para elaborar el
    histograma
76
    #lo que hace density=True es normalizar el
    histograma para que el área total sea 1, lo que permite comparar
    con la curva de densidad.
77
    # alpha ajusta la transparencia del histograma
78 # Calcular los parámetros de la distribución normal (media y
    desviación estándar)
79 mu_van, std_van = norm.fit(df['Ia']) #Con esto, lo que hacemos es
    ajustar una distribución normal a los datos dados. Recordemos que
    una distribución normal contiene
80
    # la media y la desviación estandar.
```

```
81 # Crear una serie de valores para la curva de la distribución
82 xmin_van, xmax_van = ax1.get_xlim() #lo que hace esta función es
    obtener los límites del eje X del histograma.
83 x_van = np.linspace(xmin_van, xmax_van, 100) # Usamos linspace para
    crear un intervalo entre xmin y xmax separados por 100 puntos
84 p_van = norm.pdf(x_van, mu_van, std_van) # Esto calcula la función
    de densidad de probabilidad de la distribución normal para cada
    punto en x usando los parámetros ajustados.
85
86 # Graficamos la curva de distribución
87 ax1.plot(x_van, p_van, 'k', linewidth=2) # k nos da el color de la
    curva y linewidth el ancho de la curva
88 ax1.set_xlabel('Ia')
89 ax1.set_ylabel('Densidad de probabilidad')
90 ax1.set_title('Histograma Ia')
91
92 # Histograma de 'Ib'
93
94 ax2 = axs[0,1] #establecemos que el segundo histograma tome la
    segunda posición de nuestro arreglo gráfico
95 ax2 = df['Ib'].hist(density=True, alpha=0.5, color='orange',
    edgecolor='black', ax=ax2)
96 mu_vbn, std_vbn = norm.fit(df['Ib'])
97 xmin_vbn, xmax_vbn = ax2.get_xlim()
98 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100)
99 p_vbn = norm.pdf(x_vbn, mu_vbn, std_vbn)
100 ax2.plot(x_vbn, p_vbn, 'k', linewidth=2)
101 ax2.set_xlabel('Ib')
102 ax2.set_ylabel('Densidad de probabilidad')
103 ax2.set_title('Histograma Ib')
104
105 # Histograma de 'Ic'
106
107 ax3 = axs[1,0] #establecemos que el tercer histograma tome la
    tercera posición de nuestro arreglo gráfico
108 ax3 = df['Ic'].hist(density=True, alpha=0.5, color='green',
    edgecolor='black', ax=ax3)
109 mu_vcn, std_vcn = norm.fit(df['Ic'])
110 xmin_vcn, xmax_vcn = ax3.get_xlim()
111 x_vcn = np.linspace(xmin_vcn, xmax_vcn, 100)
112 p_vcn = norm.pdf(x_vcn, mu_vcn, std_vcn)
113 ax3.plot(x_vcn, p_vcn, 'k', linewidth=2)
114 ax3.set_xlabel('Ic')
115 ax3.set_ylabel('Densidad de probabilidad')
116 ax3.set_title('Histograma Ic')
117
118 # Histograma de 'In'
119
```

```
120 ax3 = axs[1,1] #establecemos que el tercer histograma tome la
    tercera posición de nuestro arreglo gráfico
121 ax3 = df['In'].hist(density=True, alpha=0.5, color='r', edgecolor='
    black', ax=ax3)
122 mu_vcn, std_vcn = norm.fit(df['In'])
123 xmin_vcn, xmax_vcn = ax3.get_xlim()
124 x_vcn = np.linspace(xmin_vcn, xmax_vcn, 100)
125 p_vcn = norm.pdf(x_vcn, mu_vcn, std_vcn)
126 ax3.plot(x_vcn, p_vcn, 'k', linewidth=2)
127 ax3.set_xlabel('In')
128 ax3.set_ylabel('Densidad de probabilidad')
129 ax3.set_title('Histograma In')
130
131 # Mostrar las gráficas
132 plt.tight_layout() # Ajusta el espaciado entre subgráficas
133 plt.show()
134
135 #Para el histograma Ia muestra un histograma bimodal
136 #El histograma Ib es un histograma que se asemeja a un histograma
    bimodal, aunque también se puede asemejar a un histograma bimodal
137 #El histograma Ic es un histograma bimodal
138 #El histograma In es un histograma multimodal
139
140 #Histograma de Qtot y Ptot
141 #Creamos los gráficos
142 fig, axs = plt.subplots(1, 2, figsize=(18, 6)) #Las entradas 1 y 3
    nos dicen que se trata de 1 fila, tres columnas
143
144 ax1 = axs[0] #establecemos que el primer histograma tome la primer
    posición de nuestro arreglo gráfico
145
146
147 ax1 = df['Qtot'].hist(density=True, alpha=0.5, color='blue',
    edgecolor='black', ax=ax1) #Pandas tiene una función para
    elaborar el histograma
148
    #lo que hace density=True es normalizar el
    histograma para que el área total sea 1, lo que permite comparar
    con la curva de densidad.
149
    # alpha ajusta la transparencia del histograma
150 # Calcular los parámetros de la distribución normal (media y
    desviación estándar)
151 mu_van, std_van = norm.fit(df['Qtot']) #Con esto, lo que hacemos es
    ajustar una distribución normal a los datos dados. Recordemos que
    una distribución normal contiene
152
    # la media y la desviación estandar.
153 # Crear una serie de valores para la curva de la distribución
```

```
154 xmin_van, xmax_van = ax1.get_xlim() #lo que hace esta función es
    obtener los límites del eje X del histograma.
155 x_van = np.linspace(xmin_van, xmax_van, 100) # Usamos linspace para
    crear un intervalo entre xmin y xmax separados por 100 puntos
156 p_van = norm.pdf(x_van, mu_van, std_van) # Esto calcula la función
    de densidad de probabilidad de la distribución normal para cada
    punto en x usando los parámetros ajustados.

157
158 # Graficamos la curva de distribución
159 ax1.plot(x_van, p_van, 'k', linewidth=2) # k nos da el color de la
    curva y linewidth el ancho de la curva
160 ax1.set_xlabel('Qtot')
161 ax1.set_ylabel('Densidad de probabilidad')
162 ax1.set_title('Histograma Qtot')
163
164 # Histograma de 'Ptot'
165
166 ax2 = axs[1] #establecemos que el segundo histograma tome la segunda
    posición de nuestro arreglo gráfico
167 ax2 = df['Ptot'].hist(density=True, alpha=0.5, color='orange',
    edgecolor='black', ax=ax2)
168 mu_vbn, std_vbn = norm.fit(df['Ptot'])
169 xmin_vbn, xmax_vbn = ax2.get_xlim()
170 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100)
171 p_vbn = norm.pdf(x_vbn, mu_vbn, std_vbn)
172 ax2.plot(x_vbn, p_vbn, 'k', linewidth=2)
173 ax2.set_xlabel('Ptot')
174 ax2.set_ylabel('Densidad de probabilidad')
175 ax2.set_title('Histograma Ptot')
176
177
178
179 # Mostrar las gráficas
180 plt.tight_layout() # Ajusta el espaciado entre subgráficas
181 plt.show()
182
183 #Duda, como hacemos para saber el número máximo de bins? Ejemplo.
    bins=30
184
185 #Tal vez el histograma Qtot sea un histograma izquierdo sesgado
186 #El histograma Ptot es un histograma multimodal
187
188 #Histograma FPtot, THDIa, THDIb, THIc
189 #Creamos los gráficos
190 fig, axs = plt.subplots(2, 2, figsize=(10, 8)) #Las entradas 1 y 3
    nos dicen que se trata de 1 fila, tres columnas
191
192 ax1 = axs[0,0] #establecemos que el primer histograma tome la primer
```

```
    posición de nuestro arreglo gráfico
193
194
195 ax1 = df['FPtot'].hist(density=True, alpha=0.5, color='blue',
    edgecolor='black', ax=ax1) #Pandas tiene una función para
    elaborar el histograma
196
    #lo que hace density=True es normalizar el
    histograma para que el área total sea 1, lo que permite comparar
    con la curva de densidad.
197
    # alpha ajusta la transparencia del histograma
198 # Calcular los parámetros de la distribución normal (media y
    desviación estándar)
199 mu_van, std_van = norm.fit(df['FPtot']) #Con esto, lo que hacemos es
    ajustar una distribución normal a los datos dados. Recordemos
    que una distribución normal contiene
200
    # la media y la desviación estandar.
201 # Crear una serie de valores para la curva de la distribución
202 xmin_van, xmax_van = ax1.get_xlim() #lo que hace esta función es
    obtener los límites del eje X del histograma.
203 x_van = np.linspace(xmin_van, xmax_van, 100) # Usamos linspace para
    crear un intervalo entre xmin y xmax separados por 100 puntos
204 p_van = norm.pdf(x_van, mu_van, std_van) # Esto calcula la función
    de densidad de probabilidad de la distribución normal para cada
    punto en x usando los parámetros ajustados.
205
206 # Graficamos la curva de distribución
207 ax1.plot(x_van, p_van, 'k', linewidth=2) # k nos da el color de la
    curva y linewidth el ancho de la curva
208 ax1.set_xlabel('FPtot')
209 ax1.set_ylabel('Densidad de probabilidad')
210 ax1.set_title('Histograma FPtot')
211
212 # Histograma de 'THDIa'
213
214 ax2 = axs[0,1] #establecemos que el segundo histograma tome la
    segunda posición de nuestro arreglo gráfico
215 ax2 = df['THDIa'].hist(density=True, alpha=0.5, color='orange',
    edgecolor='black', ax=ax2)
216 mu_vbn, std_vbn = norm.fit(df['THDIa'])
217 xmin_vbn, xmax_vbn = ax2.get_xlim()
218 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100)
219 p_vbn = norm.pdf(x_vbn, mu_vbn, std_vbn)
220 ax2.plot(x_vbn, p_vbn, 'k', linewidth=2)
221 ax2.set_xlabel('THDIa')
222 ax2.set_ylabel('Densidad de probabilidad')
223 ax2.set_title('Histograma THDIa')
```

```
224
225 # Histograma de 'THDIb'
226
227 ax3 = axs[1,0] #establecemos que el tercer histograma tome la
                tercera posición de nuestro arreglo gráfico
228 ax3 = df['THDIb'].hist(density=True, alpha=0.5, color='green',
                edgecolor='black', ax=ax3)
229 mu_vcn, std_vcn = norm.fit(df['THDIb'])
230 xmin_vcn, xmax_vcn = ax3.get_xlim()
231 x_vcn = np.linspace(xmin_vcn, xmax_vcn, 100)
232 p_vcn = norm.pdf(x_vcn, mu_vcn, std_vcn)
233 ax3.plot(x_vcn, p_vcn, 'k', linewidth=2)
234 ax3.set_xlabel('THDIb')
235 ax3.set_ylabel('Densidad de probabilidad')
236 ax3.set_title('Histograma THDIb')
237
238 # Histograma de 'THDIc'
239
240 ax3 = axs[1,1] #establecemos que el tercer histograma tome la
                tercera posición de nuestro arreglo gráfico
241 ax3 = df['THDIc'].hist(density=True, alpha=0.5, color='r', edgecolor
                ='black', ax=ax3)
242 mu_vcn, std_vcn = norm.fit(df['THDIc'])
243 xmin_vcn, xmax_vcn = ax3.get_xlim()
244 x_vcn = np.linspace(xmin_vcn, xmax_vcn, 100)
245 p_vcn = norm.pdf(x_vcn, mu_vcn, std_vcn)
246 ax3.plot(x_vcn, p_vcn, 'k', linewidth=2)
247 ax3.set_xlabel('THDIc')
248 ax3.set_ylabel('Densidad de probabilidad')
249 ax3.set_title('Histograma THDIc')
250
251 # Mostrar las gráficas
252 plt.tight_layout() # Ajusta el espaciado entre subgráficas
253 plt.show()
254
255 #El histograma Fptot no presenta ningún tipo de distribución e
                incluso más que histograma considero que es un gráfico de barras
256 #El histograma THDIa es un histograma multimodal y parece que sigue
                una distribución normal
257 #El histograma THDIb tiende más a ser un histograma simétrico (
                chulada de histograma jajaja) y presenta una distribución normal
258 #El último histograma es multimodal
259
260 #Histograma THDVan, THDVbn, THDVcn
261 #Creamos los gráficos
262 fig, axs = plt.subplots(1, 3, figsize=(18, 6)) #Las entradas 1 y 3
                nos dicen que se trata de 1 fila, tres columnas
263
```

```
264 ax1 = axs[0] #establecemos que el primer histograma tome la primer
    posición de nuestro arreglo gráfico
265
266
267 ax1 = df['THDVan'].hist(density=True, alpha=0.5, color='blue',
    edgecolor='black', ax=ax1) #Pandas tiene una función para
    elaborar el histograma
268
    #lo que hace density=True es normalizar el
    histograma para que el área total sea 1, lo que permite comparar
    con la curva de densidad.
269
    # alpha ajusta la transparencia del histograma
270 # Calcular los parámetros de la distribución normal (media y
    desviación estándar)
271 mu_van, std_van = norm.fit(df['THDVan']) #Con esto, lo que hacemos
    es ajustar una distribución normal a los datos dados. Recordemos
    que una distribución normal contiene
272
    # la media y la desviación estandar.
273 # Crear una serie de valores para la curva de la distribución
274 xmin_van, xmax_van = ax1.get_xlim() #lo que hace esta función es
    obtener los límites del eje X del histograma.
275 x_van = np.linspace(xmin_van, xmax_van, 100) # Usamos linspace para
    crear un intervalo entre xmin y xmax separados por 100 puntos
276 p_van = norm.pdf(x_van, mu_van, std_van) # Esto calcula la función
    de densidad de probabilidad de la distribución normal para cada
    punto en x usando los parámetros ajustados.
277
278 # Graficamos la curva de distribución
279 ax1.plot(x_van, p_van, 'k', linewidth=2) # k nos da el color de la
    curva y linewidth el ancho de la curva
280 ax1.set_xlabel('THDVan')
281 ax1.set_ylabel('Densidad de probabilidad')
282 ax1.set_title('Histograma THDVan')
283
284 # Histograma de 'THDVbn'
285
286 ax2 = axs[1] #establecemos que el segundo histograma tome la segunda
    posición de nuestro arreglo gráfico
287 ax2 = df['THDVbn'].hist(density=True, alpha=0.5, color='orange',
    edgecolor='black', ax=ax2)
288 mu_vbn, std_vbn = norm.fit(df['THDVbn'])
289 xmin_vbn, xmax_vbn = ax2.get_xlim()
290 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100)
291 p_vbn = norm.pdf(x_vbn, mu_vbn, std_vbn)
292 ax2.plot(x_vbn, p_vbn, 'k', linewidth=2)
293 ax2.set_xlabel('THDVbn')
294 ax2.set_ylabel('Densidad de probabilidad')
```

```
295 ax2.set_title('Histograma THDVbn')
296
297 # Histograma de 'THDVcn'
298
299 ax3 = axs[2] #establecemos que el tercer histograma tome la tercera
    posición de nuestro arreglo gráfico
300 ax3 = df['THDVcn'].hist(density=True, alpha=0.5, color='green',
    edgecolor='black', ax=ax3)
301 mu_vcn, std_vcn = norm.fit(df['THDVcn'])
302 xmin_vcn, xmax_vcn = ax3.get_xlim()
303 x_vcn = np.linspace(xmin_vcn, xmax_vcn, 100)
304 p_vcn = norm.pdf(x_vcn, mu_vcn, std_vcn)
305 ax3.plot(x_vcn, p_vcn, 'k', linewidth=2)
306 ax3.set_xlabel('THDVcn')
307 ax3.set_ylabel('Densidad de probabilidad')
308 ax3.set_title('Histograma THDVcn')
309
310 # Mostrar las gráficas
311 plt.tight_layout() # Ajusta el espaciado entre subgráficas
312 plt.show()
313
314 #Duda, como hacemos para saber el número máximo de bins? Ejemplo.
    bins=30
315
316 #Los tres histogramas son multimodales, aunque el histograma THDVbn
    está raro, hay falta de datos.
317
318 #Histograma FPA, FPB, FPC
319 #Creamos los gráficos
320 fig, axs = plt.subplots(1, 3, figsize=(18, 6)) #Las entradas 1 y 3
    nos dicen que se trata de 1 fila, tres columnas
321
322 ax1 = axs[0] #establecemos que el primer histograma tome la primer
    posición de nuestro arreglo gráfico
323
324
325 ax1 = df['FPA'].hist(density=True, alpha=0.5, color='blue',
    edgecolor='black', ax=ax1) #Pandas tiene una función para
    elaborar el histograma
326
    #lo que hace density=True es normalizar el
    histograma para que el área total sea 1, lo que permite comparar
    con la curva de densidad.
327
    # alpha ajusta la transparencia del histograma
328 # Calcular los parámetros de la distribución normal (media y
    desviación estándar)
329 mu_van, std_van = norm.fit(df['FPA']) #Con esto, lo que hacemos es
```

```
    ajustar una distribución normal a los datos dados. Recordemos que
    una distribución normal contiene
330                                     # la media y la desviación estandar.
331 # Crear una serie de valores para la curva de la distribución
332 xmin_van, xmax_van = ax1.get_xlim() #lo que hace esta función es
    obtener los límites del eje X del histograma.
333 x_van = np.linspace(xmin_van, xmax_van, 100) # Usamos linspace para
    crear un intervalo entre xmin y xmax separados por 100 puntos
334 p_van = norm.pdf(x_van, mu_van, std_van) # Esto calcula la función
    de densidad de probabilidad de la distribución normal para cada
    punto en x usando los parámetros ajustados.
335
336 # Graficamos la curva de distribución
337 ax1.plot(x_van, p_van, 'k', linewidth=2) # k nos da el color de la
    curva y linewidth el ancho de la curva
338 ax1.set_xlabel('FPa')
339 ax1.set_ylabel('Densidad de probabilidad')
340 ax1.set_title('Histograma FPa')
341
342 # Histograma de 'FPb'
343
344 ax2 = axs[1] #establecemos que el segundo histograma tome la segunda
    posición de nuestro arreglo gráfico
345 ax2 = df['FPb'].hist(density=True, alpha=0.5, color='orange',
    edgecolor='black', ax=ax2)
346 mu_vbn, std_vbn = norm.fit(df['FPb'])
347 xmin_vbn, xmax_vbn = ax2.get_xlim()
348 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100)
349 p_vbn = norm.pdf(x_vbn, mu_vbn, std_vbn)
350 ax2.plot(x_vbn, p_vbn, 'k', linewidth=2)
351 ax2.set_xlabel('FPb')
352 ax2.set_ylabel('Densidad de probabilidad')
353 ax2.set_title('Histograma FPb')
354
355 # Histograma de 'FPc'
356
357 ax3 = axs[2] #establecemos que el tercer histograma tome la tercera
    posición de nuestro arreglo gráfico
358 ax3 = df['FPc'].hist(density=True, alpha=0.5, color='green',
    edgecolor='black', ax=ax3)
359 mu_vcn, std_vcn = norm.fit(df['FPc'])
360 xmin_vcn, xmax_vcn = ax3.get_xlim()
361 x_vcn = np.linspace(xmin_vcn, xmax_vcn, 100)
362 p_vcn = norm.pdf(x_vcn, mu_vcn, std_vcn)
363 ax3.plot(x_vcn, p_vcn, 'k', linewidth=2)
364 ax3.set_xlabel('FPc')
365 ax3.set_ylabel('Densidad de probabilidad')
366 ax3.set_title('Histograma FPc')
```

```
367
368 # Mostrar las gráficas
369 plt.tight_layout() # Ajusta el espaciado entre subgráficas
370 plt.show()
371
372 ##### OUTLIERS DE LOS DIAGRAMAS VAN,VBN,VCN
373 #####
374
375 # Crear la figura y los subgráficos
376 fig, axs = plt.subplots(1, 3, figsize=(12, 6))
377
378 # Subgráfico 1 (Van)
379 ax1 = axs[0]
380 ax1.hist(df['Van'], density=True, alpha=0.5, color='blue', edgecolor
381         ='black')
382
383 aX1 = df['Van'].dropna().values.reshape(-1, 1)
384 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(aX1)
385 kde_scores = kde.score_samples(aX1)
386 kde_scores_series = pd.Series(kde_scores)
387
388 # Cálculo de valores atípicos
389 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
390 iqr = q3 - q1
391 threshold = q1 - (2.2 * iqr)
392 rare_values = df['Van'][kde_scores < threshold].tolist()
393 print("Valores atípicos detectados en Van:", rare_values)
394
395 # Estimación de densidad
396 xmin_van, xmax_van = ax1.get_xlim()
397 x_van = np.linspace(xmin_van, xmax_van, 100).reshape(-1, 1)
398 p_van = np.exp(kde.score_samples(x_van))
399 ax1.plot(x_van, p_van, 'k', linewidth=2)
400
401 # Dibujar líneas para valores raros
402 for rare_value in rare_values:
403     ax1.axvline(rare_value, color='red', linestyle='-.')
404
405 ax1.set_xlabel('Van')
406 ax1.set_ylabel('Densidad de probabilidad')
407 ax1.set_title('Histograma Van con KDE y valores raros')
408
409 # Subgráfico 2 (Vbn)
410 ax2 = axs[1]
411 ax2.hist(df['Vbn'], density=True, alpha=0.5, color='blue', edgecolor
412         ='black')
413
414 aX2 = df['Vbn'].dropna().values.reshape(-1, 1)
```

```
412 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(aX2)
413 kde_scores = kde.score_samples(aX2)
414 kde_scores_series = pd.Series(kde_scores)
415
416 # Cálculo de valores atípicos
417 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
418 iqr = q3 - q1
419 threshold = q1 - (2.2 * iqr)
420 rare_values = df['Vbn'][kde_scores < threshold].tolist()
421 print("Valores atípicos detectados en Vbn:", rare_values)
422
423 # Estimación de densidad
424 xmin_vbn, xmax_vbn = ax2.get_xlim()
425 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100).reshape(-1, 1)
426 p_vbn = np.exp(kde.score_samples(x_vbn))
427 ax2.plot(x_vbn, p_vbn, 'k', linewidth=2)
428
429 # Dibujar líneas para valores raros
430 for rare_value in rare_values:
431     ax2.axvline(rare_value, color='red', linestyle='-.')
432
433 ax2.set_xlabel('Vbn')
434 ax2.set_ylabel('Densidad de probabilidad')
435 ax2.set_title('Histograma Vbn con KDE y outliers')
436
437 # Subgráfico 3 (Vcn)
438 ax3 = axs[2]
439 ax3.hist(df['Vcn'], density=True, alpha=0.5, color='blue', edgecolor
         = 'black')
440
441 aX3 = df['Vcn'].dropna().values.reshape(-1, 1)
442 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(aX3)
443 kde_scores = kde.score_samples(aX3)
444 kde_scores_series = pd.Series(kde_scores)
445
446 # Cálculo de valores atípicos
447 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
448 iqr = q3 - q1
449 threshold = q1 - (2.2 * iqr)
450 rare_values = df['Vcn'][kde_scores < threshold].tolist()
451 print("Valores atípicos detectados en Vcn:", rare_values)
452
453 # Estimación de densidad
454 xmin_vbn, xmax_vbn = ax2.get_xlim()
455 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100).reshape(-1, 1)
456 p_vbn = np.exp(kde.score_samples(x_vbn))
457 ax3.plot(x_vbn, p_vbn, 'k', linewidth=2)
458
```

```
459 # Dibujar líneas para valores raros
460 for rare_value in rare_values:
461     ax3.axvline(rare_value, color='red', linestyle='-.')
462
463 ax3.set_xlabel('Vcn')
464 ax3.set_ylabel('Densidad de probabilidad')
465 ax3.set_title('Histograma Vcn con KDE y outliers')
466
467 # Ajustar el espaciado entre subgráficas
468 plt.tight_layout()
469 plt.show()
470
471 ##### OUTLIERS DE LAS CORIENTES
472 #####
473
474 # Crear la figura y los subgráficos
475 fig, axs = plt.subplots(2, 2, figsize=(10, 8))
476
477 # Subgráfico 1 (Van)
478 ax1 = axs[0,0]
479 ax1.hist(df['Ia'], density=True, alpha=0.5, color='blue', edgecolor=
480         'black')
481
482 aX1 = df['Ia'].dropna().values.reshape(-1, 1)
483 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(aX1)
484 kde_scores = kde.score_samples(aX1)
485 kde_scores_series = pd.Series(kde_scores)
486
487 # Cálculo de valores atípicos
488 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
489 iqr = q3 - q1
490 threshold = q1 - (2.2 * iqr)
491 rare_values = df['Ia'][kde_scores < threshold].tolist()
492 print("Valores atípicos detectados en Ia:", rare_values)
493
494 # Estimación de densidad
495 xmin_van, xmax_van = ax1.get_xlim()
496 x_van = np.linspace(xmin_van, xmax_van, 100).reshape(-1, 1)
497 p_van = np.exp(kde.score_samples(x_van))
498 ax1.plot(x_van, p_van, 'k', linewidth=2)
499
500 # Dibujar líneas para valores raros
501 for rare_value in rare_values:
502     ax1.axvline(rare_value, color='red', linestyle='-.')
503
504 ax1.set_xlabel('Ia')
505 ax1.set_ylabel('Densidad de probabilidad')
506 ax1.set_title('Histograma Ia con KDE y valores raros')
```

```
505
506 # Subgráfico 2 (Ib)
507 ax2 = axs[0,1]
508 ax2.hist(df['Ib'], density=True, alpha=0.5, color='orange',
          edgecolor='black')
509
510 aX2 = df['Ib'].dropna().values.reshape(-1, 1)
511 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(aX2)
512 kde_scores = kde.score_samples(aX2)
513 kde_scores_series = pd.Series(kde_scores)
514
515 # Cálculo de valores atípicos
516 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
517 iqr = q3 - q1
518 threshold = q1 - (2.2 * iqr)
519 rare_values = df['Ib'][kde_scores < threshold].tolist()
520 print("Valores atípicos detectados en Ib:", rare_values)
521
522 # Estimación de densidad
523 xmin_vbn, xmax_vbn = ax2.get_xlim()
524 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100).reshape(-1, 1)
525 p_vbn = np.exp(kde.score_samples(x_vbn))
526 ax2.plot(x_vbn, p_vbn, 'k', linewidth=2)
527
528 # Dibujar líneas para valores raros
529 for rare_value in rare_values:
530     ax2.axvline(rare_value, color='red', linestyle='-.')
531
532 ax2.set_xlabel('Ib')
533 ax2.set_ylabel('Densidad de probabilidad')
534 ax2.set_title('Histograma Ib con KDE y outliers')
535
536 # Subgráfico 3 (Ic)
537 ax3 = axs[1,0]
538 ax3.hist(df['Ic'], density=True, alpha=0.5, color='green', edgecolor
          = 'black')
539
540 aX3 = df['Ic'].dropna().values.reshape(-1, 1)
541 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(aX3)
542 kde_scores = kde.score_samples(aX3)
543 kde_scores_series = pd.Series(kde_scores)
544
545 # Cálculo de valores atípicos
546 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
547 iqr = q3 - q1
548 threshold = q1 - (2.2 * iqr)
549 rare_values = df['Ic'][kde_scores < threshold].tolist()
550 print("Valores atípicos detectados en Ic:", rare_values)
```

```
551
552 # Estimación de densidad
553 xmin_vbn, xmax_vbn = ax3.get_xlim()
554 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100).reshape(-1, 1)
555 p_vbn = np.exp(kde.score_samples(x_vbn))
556 ax3.plot(x_vbn, p_vbn, 'k', linewidth=2)
557
558 # Dibujar líneas para valores raros
559 for rare_value in rare_values:
560     ax3.axvline(rare_value, color='red', linestyle='-.')
561
562 ax3.set_xlabel('Ic')
563 ax3.set_ylabel('Densidad de probabilidad')
564 ax3.set_title('Histograma Ic con KDE y outliers')
565
566 # Subgráfico 4 (In)
567 ax4 = axs[1,1]
568 ax4.hist(df['In'], density=True, alpha=0.5, color='red', edgecolor='
    black')
569
570 aX4 = df['In'].dropna().values.reshape(-1, 1)
571 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(aX4)
572 kde_scores = kde.score_samples(aX4)
573 kde_scores_series = pd.Series(kde_scores)
574
575 # Cálculo de valores atípicos
576 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
577 iqr = q3 - q1
578 threshold = q1 - (2.2 * iqr)
579 rare_values = df['In'][kde_scores < threshold].tolist()
580 print("Valores atípicos detectados en In:", rare_values)
581
582 # Estimación de densidad
583 xmin_vbn, xmax_vbn = ax4.get_xlim()
584 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100).reshape(-1, 1)
585 p_vbn = np.exp(kde.score_samples(x_vbn))
586 ax4.plot(x_vbn, p_vbn, 'k', linewidth=2)
587
588 # Dibujar líneas para valores raros
589 for rare_value in rare_values:
590     ax4.axvline(rare_value, color='black', linestyle='-.')
591
592 ax4.set_xlabel('In')
593 ax4.set_ylabel('Densidad de probabilidad')
594 ax4.set_title('Histograma In con KDE y outliers')
595
596 # Ajustar el espaciado entre subgráficas
597 plt.tight_layout()
```

```
598 plt.show()
599
600
601 ##### Outliers Qtot y Ptot #####
602
603 # Crear la figura y los subgráficos
604 fig, axs = plt.subplots(1, 2, figsize=(18, 6))
605
606 # Subgráfico 1 (Qtot)
607 ax1 = axs[0]
608 ax1.hist(df['Qtot'], density=True, alpha=0.5, color='blue',
609          edgecolor='black')
610
611 aX1 = df['Qtot'].dropna().values.reshape(-1, 1)
612 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(aX1)
613 kde_scores = kde.score_samples(aX1)
614 kde_scores_series = pd.Series(kde_scores)
615
616 # Cálculo de valores atípicos
617 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
618 iqr = q3 - q1
619 threshold = q1 - (2.2 * iqr)
620 rare_values = df['Qtot'][kde_scores < threshold].tolist()
621 print("Valores atípicos detectados en Qtot:", rare_values)
622
623 # Estimación de densidad
624 xmin_van, xmax_van = ax1.get_xlim()
625 x_van = np.linspace(xmin_van, xmax_van, 100).reshape(-1, 1)
626 p_van = np.exp(kde.score_samples(x_van))
627 ax1.plot(x_van, p_van, 'k', linewidth=2)
628
629 # Dibujar líneas para valores raros
630 for rare_value in rare_values:
631     ax1.axvline(rare_value, color='red', linestyle='-.')
632
633 ax1.set_xlabel('Qtot')
634 ax1.set_ylabel('Densidad de probabilidad')
635 ax1.set_title('Histograma Qtot con KDE y valores raros')
636
637 # Subgráfico 2 (Ptot)
638 ax2 = axs[1]
639 ax2.hist(df['Ptot'], density=True, alpha=0.5, color='orange',
640          edgecolor='black')
641
642 aX2 = df['Ptot'].dropna().values.reshape(-1, 1)
643 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(aX2)
644 kde_scores = kde.score_samples(aX2)
645 kde_scores_series = pd.Series(kde_scores)
```

```
644
645 # Cálculo de valores atípicos
646 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
647 iqr = q3 - q1
648 threshold = q1 - (2.2 * iqr)
649 rare_values = df['Ptot'][kde_scores < threshold].tolist()
650 print("Valores atípicos detectados en Ptot:", rare_values)
651
652 # Estimación de densidad
653 xmin_vbn, xmax_vbn = ax2.get_xlim()
654 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100).reshape(-1, 1)
655 p_vbn = np.exp(kde.score_samples(x_vbn))
656 ax2.plot(x_vbn, p_vbn, 'k', linewidth=2)
657
658 # Dibujar líneas para valores raros
659 for rare_value in rare_values:
660     ax2.axvline(rare_value, color='red', linestyle='-.')
661
662 ax2.set_xlabel('Ptot')
663 ax2.set_ylabel('Densidad de probabilidad')
664 ax2.set_title('Histograma Ptot con KDE y outliers')
665
666 plt.tight_layout() # Ajusta el espaciado entre subgráficas
667 plt.show()
668
669 ##### Outliers FPtot, THDIa, THDIb, THDIc #####
670
671 # Crear la figura y los subgráficos
672 fig, axs = plt.subplots(2, 2, figsize=(10, 8))
673
674 # Subgráfico 1 (FPtot)
675 ax1 = axs[0,0]
676 ax1.hist(df['FPtot'], density=True, alpha=0.5, color='blue',
677          edgecolor='black')
678
679 ax1 = df['FPtot'].dropna().values.reshape(-1, 1)
680 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(ax1)
681 kde_scores = kde.score_samples(ax1)
682 kde_scores_series = pd.Series(kde_scores)
683
684 # Cálculo de valores atípicos
685 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
686 iqr = q3 - q1
687 threshold = q1 - (2.2 * iqr)
688 rare_values = df['FPtot'][kde_scores < threshold].tolist()
689 print("Valores atípicos detectados en FPtot:", rare_values)
690
691 # Estimación de densidad
```

```
691 xmin_van, xmax_van = ax1.get_xlim()
692 x_van = np.linspace(xmin_van, xmax_van, 100).reshape(-1, 1)
693 p_van = np.exp(kde.score_samples(x_van))
694 ax1.plot(x_van, p_van, 'k', linewidth=2)
695
696 # Dibujar líneas para valores raros
697 for rare_value in rare_values:
698     ax1.axvline(rare_value, color='red', linestyle='-.')
699
700 ax1.set_xlabel('FPtot')
701 ax1.set_ylabel('Densidad de probabilidad')
702 ax1.set_title('Histograma FPtot con KDE y valores raros')
703
704 # Subgráfico 2 (THDIa)
705 ax2 = axs[0,1]
706 ax2.hist(df['THDIa'], density=True, alpha=0.5, color='orange',
707          edgecolor='black')
708
709 ax2 = df['THDIa'].dropna().values.reshape(-1, 1)
710 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(ax2)
711 kde_scores = kde.score_samples(ax2)
712 kde_scores_series = pd.Series(kde_scores)
713
714 # Cálculo de valores atípicos
715 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
716 iqr = q3 - q1
717 threshold = q1 - (2.2 * iqr)
718 rare_values = df['THDIa'][kde_scores < threshold].tolist()
719 print("Valores atípicos detectados en THDIa:", rare_values)
720
721 # Estimación de densidad
722 xmin_vbn, xmax_vbn = ax2.get_xlim()
723 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100).reshape(-1, 1)
724 p_vbn = np.exp(kde.score_samples(x_vbn))
725 ax2.plot(x_vbn, p_vbn, 'k', linewidth=2)
726
727 # Dibujar líneas para valores raros
728 for rare_value in rare_values:
729     ax2.axvline(rare_value, color='red', linestyle='-.')
730
731 ax2.set_xlabel('THDIa')
732 ax2.set_ylabel('Densidad de probabilidad')
733 ax2.set_title('Histograma THDIa con KDE y outliers')
734
735 # Subgráfico 3 (THDIb)
736 ax3 = axs[1,0]
737 ax3.hist(df['THDIb'], density=True, alpha=0.5, color='green',
738          edgecolor='black')
```

```
737
738 ax3 = df['THDIb'].dropna().values.reshape(-1, 1)
739 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(ax3)
740 kde_scores = kde.score_samples(ax3)
741 kde_scores_series = pd.Series(kde_scores)
742
743 # Cálculo de valores atípicos
744 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
745 iqr = q3 - q1
746 threshold = q1 - (2.2 * iqr)
747 rare_values = df['THDIb'][kde_scores < threshold].tolist()
748 print("Valores atípicos detectados en THDIb:", rare_values)
749
750 # Estimación de densidad
751 xmin_vbn, xmax_vbn = ax3.get_xlim()
752 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100).reshape(-1, 1)
753 p_vbn = np.exp(kde.score_samples(x_vbn))
754 ax3.plot(x_vbn, p_vbn, 'k', linewidth=2)
755
756 # Dibujar líneas para valores raros
757 for rare_value in rare_values:
758     ax3.axvline(rare_value, color='red', linestyle='-.')
759
760 ax3.set_xlabel('THDIb')
761 ax3.set_ylabel('Densidad de probabilidad')
762 ax3.set_title('Histograma THDIb con KDE y outliers')
763
764 # Subgráfico 4 (THDIc)
765 ax4 = axs[1,1]
766 ax4.hist(df['THDIc'], density=True, alpha=0.5, color='red',
767         edgecolor='black')
768
769 ax4 = df['THDIc'].dropna().values.reshape(-1, 1)
770 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(ax4)
771 kde_scores = kde.score_samples(ax4)
772 kde_scores_series = pd.Series(kde_scores)
773
774 # Cálculo de valores atípicos
775 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
776 iqr = q3 - q1
777 threshold = q1 - (2.2 * iqr)
778 rare_values = df['THDIc'][kde_scores < threshold].tolist()
779 print("Valores atípicos detectados en THDIc:", rare_values)
780
781 # Estimación de densidad
782 xmin_vbn, xmax_vbn = ax4.get_xlim()
783 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100).reshape(-1, 1)
784 p_vbn = np.exp(kde.score_samples(x_vbn))
```

```
784 ax4.plot(x_vbn, p_vbn, 'k', linewidth=2)
785
786 # Dibujar líneas para valores raros
787 for rare_value in rare_values:
788     ax4.axvline(rare_value, color='black', linestyle='-.')
789
790 ax4.set_xlabel('THDIc')
791 ax4.set_ylabel('Densidad de probabilidad')
792 ax4.set_title('Histograma THDIc con KDE y outliers')
793
794 # Ajustar el espaciado entre subgráficas
795 plt.tight_layout()
796 plt.show()
797
798
799
800 ##### Outliers THDVan, THDVbn, THDVcn
801 #####
802
803 # Crear la figura y los subgráficos
804 fig, axs = plt.subplots(1, 3, figsize=(12, 6))
805
806 # Subgráfico 1 (THDVan)
807 ax1 = axs[0]
808 ax1.hist(df['THDVan'], density=True, alpha=0.5, color='blue',
809         edgecolor='black')
810
811 aX1 = df['THDVan'].dropna().values.reshape(-1, 1)
812 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(aX1)
813 kde_scores = kde.score_samples(aX1)
814 kde_scores_series = pd.Series(kde_scores)
815
816 # Cálculo de valores atípicos
817 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
818 iqr = q3 - q1
819 threshold = q1 - (2.2 * iqr)
820 rare_values = df['THDVan'][kde_scores < threshold].tolist()
821 print("Valores atípicos detectados en THDVan:", rare_values)
822
823 # Estimación de densidad
824 xmin_van, xmax_van = ax1.get_xlim()
825 x_van = np.linspace(xmin_van, xmax_van, 100).reshape(-1, 1)
826 p_van = np.exp(kde.score_samples(x_van))
827 ax1.plot(x_van, p_van, 'k', linewidth=2)
828
829 # Dibujar líneas para valores raros
830 for rare_value in rare_values:
831     ax1.axvline(rare_value, color='red', linestyle='-.')
```

```
830
831 ax1.set_xlabel('THDVan')
832 ax1.set_ylabel('Densidad de probabilidad')
833 ax1.set_title('Histograma THDVan con KDE y valores raros')
834
835 # Subgráfico 2 (THDVbn)
836 ax2 = axs[1]
837 ax2.hist(df['THDVbn'], density=True, alpha=0.5, color='green',
          edgecolor='black')
838
839 aX2 = df['THDVbn'].dropna().values.reshape(-1, 1)
840 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(aX2)
841 kde_scores = kde.score_samples(aX2)
842 kde_scores_series = pd.Series(kde_scores)
843
844 # Cálculo de valores atípicos
845 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
846 iqr = q3 - q1
847 threshold = q1 - (2.2 * iqr)
848 rare_values = df['THDVbn'][kde_scores < threshold].tolist()
849 print("Valores atípicos detectados en THDVbn:", rare_values)
850
851 # Estimación de densidad
852 xmin_vbn, xmax_vbn = ax2.get_xlim()
853 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100).reshape(-1, 1)
854 p_vbn = np.exp(kde.score_samples(x_vbn))
855 ax2.plot(x_vbn, p_vbn, 'k', linewidth=2)
856
857 # Dibujar líneas para valores raros
858 for rare_value in rare_values:
859     ax2.axvline(rare_value, color='red', linestyle='-.')
860
861 ax2.set_xlabel('THDVbn')
862 ax2.set_ylabel('Densidad de probabilidad')
863 ax2.set_title('Histograma THDVbn con KDE y outliers')
864
865 # Subgráfico 3 (THDVcn)
866 ax3 = axs[2]
867 ax3.hist(df['THDVcn'], density=True, alpha=0.5, color='orange',
          edgecolor='black')
868
869 aX3 = df['THDVcn'].dropna().values.reshape(-1, 1)
870 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(aX3)
871 kde_scores = kde.score_samples(aX3)
872 kde_scores_series = pd.Series(kde_scores)
873
874 # Cálculo de valores atípicos
875 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
```

```
876 iqr = q3 - q1
877 threshold = q1 - (2.2 * iqr)
878 rare_values = df['THDVcn'][kde_scores < threshold].tolist()
879 print("Valores atípicos detectados en THDVcn:", rare_values)
880
881 # Estimación de densidad
882 xmin_vbn, xmax_vbn = ax2.get_xlim()
883 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100).reshape(-1, 1)
884 p_vbn = np.exp(kde.score_samples(x_vbn))
885 ax3.plot(x_vbn, p_vbn, 'k', linewidth=2)
886
887 # Dibujar líneas para valores raros
888 for rare_value in rare_values:
889     ax3.axvline(rare_value, color='red', linestyle='-.')
890
891 ax3.set_xlabel('THDVcn')
892 ax3.set_ylabel('Densidad de probabilidad')
893 ax3.set_title('Histograma THDVcn con KDE y outliers')
894
895 # Ajustar el espaciado entre subgráficas
896 plt.tight_layout()
897 plt.show()
898
899 ##### Ouliers Fpa, Fpb,FPc #####
900
901 # Crear la figura y los subgráficos
902 fig, axs = plt.subplots(1, 3, figsize=(12, 6))
903
904 # Subgráfico 1 (Fpa)
905 ax1 = axs[0]
906 ax1.hist(df['Fpa'], density=True, alpha=0.5, color='blue', edgecolor
          ='black')
907
908 ax1 = df['Fpa'].dropna().values.reshape(-1, 1)
909 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(ax1)
910 kde_scores = kde.score_samples(ax1)
911 kde_scores_series = pd.Series(kde_scores)
912
913 # Cálculo de valores atípicos
914 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
915 iqr = q3 - q1
916 threshold = q1 - (2.2 * iqr)
917 rare_values = df['Fpa'][kde_scores < threshold].tolist()
918 print("Valores atípicos detectados en Fpa:", rare_values)
919
920 # Estimación de densidad
921 xmin_van, xmax_van = ax1.get_xlim()
922 x_van = np.linspace(xmin_van, xmax_van, 100).reshape(-1, 1)
```

```
923 p_van = np.exp(kde.score_samples(x_van))
924 ax1.plot(x_van, p_van, 'k', linewidth=2)
925
926 # Dibujar líneas para valores raros
927 for rare_value in rare_values:
928     ax1.axvline(rare_value, color='red', linestyle='-.')
929
930 ax1.set_xlabel('FPa')
931 ax1.set_ylabel('Densidad de probabilidad')
932 ax1.set_title('Histograma FPa con KDE y valores raros')
933
934 # Subgráfico 2 (FPb)
935 ax2 = axs[1]
936 ax2.hist(df['FPb'], density=True, alpha=0.5, color='green',
          edgecolor='black')
937
938 aX2 = df['FPb'].dropna().values.reshape(-1, 1)
939 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(aX2)
940 kde_scores = kde.score_samples(aX2)
941 kde_scores_series = pd.Series(kde_scores)
942
943 # Cálculo de valores atípicos
944 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
945 iqr = q3 - q1
946 threshold = q1 - (2.2 * iqr)
947 rare_values = df['FPb'][kde_scores < threshold].tolist()
948 print("Valores atípicos detectados en FPb:", rare_values)
949
950 # Estimación de densidad
951 xmin_vbn, xmax_vbn = ax2.get_xlim()
952 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100).reshape(-1, 1)
953 p_vbn = np.exp(kde.score_samples(x_vbn))
954 ax2.plot(x_vbn, p_vbn, 'k', linewidth=2)
955
956 # Dibujar líneas para valores raros
957 for rare_value in rare_values:
958     ax2.axvline(rare_value, color='red', linestyle='-.')
959
960 ax2.set_xlabel('FPb')
961 ax2.set_ylabel('Densidad de probabilidad')
962 ax2.set_title('Histograma FPb con KDE y outliers')
963
964 # Subgráfico 3 (FPc)
965 ax3 = axs[2]
966 ax3.hist(df['FPc'], density=True, alpha=0.5, color='orange',
          edgecolor='black')
967
968 aX3 = df['FPc'].dropna().values.reshape(-1, 1)
```

```
969 kde = KernelDensity(kernel='gaussian', bandwidth=0.2).fit(aX3)
970 kde_scores = kde.score_samples(aX3)
971 kde_scores_series = pd.Series(kde_scores)
972
973 # Cálculo de valores atípicos
974 q1, q3 = kde_scores_series.quantile([0.25, 0.75])
975 iqr = q3 - q1
976 threshold = q1 - (2.2 * iqr)
977 rare_values = df['FPc'][kde_scores < threshold].tolist()
978 print("Valores atípicos detectados en FPc:", rare_values)
979
980 # Estimación de densidad
981 xmin_vbn, xmax_vbn = ax2.get_xlim()
982 x_vbn = np.linspace(xmin_vbn, xmax_vbn, 100).reshape(-1, 1)
983 p_vbn = np.exp(kde.score_samples(x_vbn))
984 ax3.plot(x_vbn, p_vbn, 'k', linewidth=2)
985
986 # Dibujar líneas para valores raros
987 for rare_value in rare_values:
988     ax3.axvline(rare_value, color='red', linestyle='-.')
989
990 ax3.set_xlabel('FPc')
991 ax3.set_ylabel('Densidad de probabilidad')
992 ax3.set_title('Histograma FPc con KDE y outliers')
993
994 # Ajustar el espaciado entre subgráficas
995 plt.tight_layout()
996 plt.show()
```

Gráficas de corrientes con valores atípicos

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import matplotlib.dates as mdates
4 import mplcursors
5 from scipy.stats import zscore
6 import seaborn as sns
7 from matplotlib.ticker import MaxNLocator
8 from scipy.stats import median_abs_deviation
9 from scipy.stats import gaussian_kde
10 import numpy as np
11 from scipy.signal import find_peaks
12 #Método por percentiles y MAD
13
14 datos = pd.read_csv('C:\\Users\\Yahir\\Documents\\Documentos SS\\1SD
    -A1-2_MED-T1_20230218_1.csv')
```

```
15
16 datos_select = datos.iloc[:17280].dropna().drop_duplicates()
17
18 datos_select['t'] = pd.to_datetime(datos_select['t'])
19
20 columnas_corriente = [col for col in datos_select.columns if col.
    startswith('I')]
21
22 for column in columnas_corriente:
23     fig, ax = plt.subplots(figsize=(12, 6))
24
25     # Formato de tiempo en eje X
26     ax.xaxis.set_major_formatter(mdates.DateFormatter('%H:%M:%S'))
27     ax.xaxis.set_major_locator(mdates.HourLocator(interval=1))
28
29     items_interactivos = []
30     labels = False
31
32     for hora in range(24):
33         datos_hora = datos_select.iloc[hora * 720 : (hora + 1) *
720] # Filtrar datos por hora
34         t_inicio = datos_hora["t"].min()
35         t_fin = datos_hora["t"].max()
36
37
38         media_corriente = datos_hora[column].mean()
39         mediana = datos_hora[column].median()
40         mad = median_abs_deviation(datos_hora[column])
41         iqr_lower_limit = mediana - (1.5 * mad)
42         iqr_upper_limit = mediana + (1.5 * mad)
43
44         # límites usando percentiles
45         percentile_5 = datos_hora[column].quantile(0.001)
46         percentile_95 = datos_hora[column].quantile(0.999)
47
48         # Graficar datos de corriente
49         puntos = ax.plot(datos_hora["t"], datos_hora[column], marker
    ='o', linestyle='-')
50         items_interactivos.extend(puntos)
51
52
53         if not labels:
54             ax.hlines(y=media_corriente, xmin=t_inicio, xmax=t_fin,
    color='gray', linestyle='--', alpha=0.7, label="Media")
55             ax.hlines(y=iqr_lower_limit, xmin=t_inicio, xmax=t_fin,
    color='green', linestyle='dotted', alpha=0.7, label="Lower IQR")
56             ax.hlines(y=iqr_upper_limit, xmin=t_inicio, xmax=t_fin,
    color='red', linestyle='dotted', alpha=0.7, label="Upper IQR")
```

```

57
58     # Líneas de percentiles (5% y 95%)
59     ax.hlines(y=percentile_5, xmin=t_inicio, xmax=t_fin,
60             color='purple', linestyle='dashed', alpha=0.7, label="Percentil
61             5%")
62     ax.hlines(y=percentile_95, xmin=t_inicio, xmax=t_fin,
63             color='purple', linestyle='dashed', alpha=0.7, label="Percentil
64             95%")
65
66     labels = True
67     else:
68         ax.hlines(y=media_corriente, xmin=t_inicio, xmax=t_fin,
69                 color='gray', linestyle='--', alpha=0.7)
70         ax.hlines(y=iqr_lower_limit, xmin=t_inicio, xmax=t_fin,
71                 color='green', linestyle='dotted', alpha=0.7)
72         ax.hlines(y=iqr_upper_limit, xmin=t_inicio, xmax=t_fin,
73                 color='red', linestyle='dotted', alpha=0.7)
74
75     # Límites de percentiles
76     ax.hlines(y=percentile_5, xmin=t_inicio, xmax=t_fin,
77             color='purple', linestyle='dashed', alpha=0.7)
78     ax.hlines(y=percentile_95, xmin=t_inicio, xmax=t_fin,
79             color='purple', linestyle='dashed', alpha=0.7)
80
81     cursor = mplcursors.cursor(items_interactivos, hover=True)
82     cursor.connect("add", lambda sel: sel.annotation.set_text(f"{sel
83     .target[1]:.2f}"))
84
85     plt.xticks(rotation=45)
86     ax.set_xlabel("Tiempo (Hora:Minuto:Segundo)")
87     ax.set_ylabel("Corriente (I)")
88     ax.set_title(f"Corriente {column} 20230218")
89     ax.legend()
90     ax.grid(True)
91     plt.show()

```

Detección de outliers

```

1
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import matplotlib.dates as mdates
5 import mplcursors
6 import numpy as np
7

```

```
8 # Cargar datos
9 datos = pd.read_csv('C:\\Users\\Yahir\\Documents\\Documentos SS\\1SD
    -A1-2_MED-T1_20230218_1.csv')
10
11
12 # Preprocesamiento
13 datos_select = datos.iloc[:17280].dropna().drop_duplicates()
14 datos_select['t'] = pd.to_datetime(datos_select['t'])
15
16 # Seleccionar columnas de corriente
17 columnas_corriente = [col for col in datos_select.columns if col.
    startswith('I')]
18
19 # Creamos un DataFrame vacío para almacenar los outliers detectados.
20 outliers_global = pd.DataFrame()
21
22 # Iniciamos el análisis por hora para cada corriente
23 for column in columnas_corriente:
24     fig, ax = plt.subplots(figsize=(12, 6))
25     ax.xaxis.set_major_formatter(mdates.DateFormatter('%H:%M:%S'))
26     ax.xaxis.set_major_locator(mdates.HourLocator(interval=1)) #En
    estas líneas creamos una figura para la gráfica y configuramos el
    formato del eje X
27     #para que muestre las horas en formato.
28     #Esta última línea está indicando que las marcas (ticks) en el
    eje X deben aparecer cada 1 hora, eso se hace con (interval=1)
29
30     items_interactivos = [] #con esto creamos una lista con items
    con los cuales queremos interactuar. Con mplcursors haremos
    llamar a esta lista más adelante.
31     labels = False #Esta línea coloca las etiquetas en la gráfica.
    Después, en los siguientes ciclos (para las demás horas o
    columnas), labels se cambia a True,
32     #lo que evita que las etiquetas se repitan.
33
34     # DataFrame para almacenar datos limpios.
35     #Aquí se está creando un DataFrame vacío con el nombre
    datos_limpios. Este DataFrame se utilizará para almacenar los
    datos que han sido limpiados
36     #en cada iteración del bucle posterior.
37     datos_limpios = pd.DataFrame()
38
39     for hora in range(24):
40         datos_hora = datos_select.iloc[hora * 720 : (hora + 1) *
    720]
41         if datos_hora.empty: #Si el DataFrame datos_hora está vacío,
            la instrucción continue hace que el bucle pase a la siguiente
            iteración
```

```
42         #(es decir, pasa a la siguiente hora), sin ejecutar el c
        ódigo que sigue dentro del bucle para esa hora.
43         continue # Por si no hay suficientes datos
44
45         t_inicio = datos_hora["t"].min()
46         t_fin = datos_hora["t"].max() #Calculamos los tiempos de
        inicio y fin en esa hora específica.
47
48         #Detección de outliers usando como referencia los
        percentiles
49         low=0.001
50         high=0.999
51         percentile_low = datos_hora[column].quantile(low)
52         percentile_high = datos_hora[column].quantile(high)
53         es_outlier = (datos_hora[column] < percentile_low) | (
        datos_hora[column] > percentile_high)
54
55         # Con esto guardarmoa outliers con su timestamp
56         outliers_hora = datos_hora[es_outlier].copy() #Se están
        seleccionando los datos que son outliers dentro de datos_hora y
        hace una copia de esos datos seleccionados
57         #para evitar problemas al modificar luego el DataFrame.
58         outliers_hora['Hora'] = hora #Agrega una nueva columna
        llamada "Hora" al DataFrame outliers_hora. De esta forma podemos
        saber a qué hora del día pertenece cada outlier.
59         outliers_hora['Variable'] = column #Agrega otra nueva
        columna llamada "Variable" al DataFrame outliers_hora. Ahí se
        guarda el nombre de la variable
60         #(column) que se está analizando en ese momento
61         outliers_global = pd.concat([outliers_global, outliers_hora
        ])#Une esatos dos DataFrame al DataFrame global de los otuliers.
62         #outliers_global[outliers_global['Variable'] == 'Ia'][['t',
        'Ia']]
63         #print(outliers_global)
64
65         #for index, row in outliers_global.iterrows():
66             #variable = row['Variable'] # Qué variable fue anó
        mala ('Ia', 'Ib', etc.)
67             #valor_outlier = row[variable] # El valor real anómalo
        en esa variable
68             #tiempo = row['t'] # El momento en que
        ocurrió
69             #print(f"Outlier en {variable}: valor = {valor_outlier}
        a las {tiempo}")
70
71         outliers_en_hora = outliers_global[outliers_global['Hora']
        == hora]
72         if not outliers_en_hora.empty:
```

```
73         for _, row in outliers_en_hora.iterrows():
74             variable = row['Variable']
75             valor_outlier = row[variable]
76             tiempo = row['t']
77             print(f"Outlier en {variable}: valor = {
valor_outlier} a las {tiempo}")
78
79         #Eliminar outliers (reemplazar con NaN)
80         datos_hora_limpios = datos_hora.copy() #Hacemos una copia
del DataFrame datos_hora para no modificar directamente los datos
originales cuando
81         #trabajemos sobre ellos.
82         datos_hora_limpios.loc[es_outlier, column] = np.nan #Busca
todos los valores de la columna que son outliers (es_outlier ==
True).
83
84         #Interpolamos linealmente para rellenar valores faltantes
85         datos_hora_limpios[column] = datos_hora_limpios[column].
interpolate(method='linear') #Esto toma la columna column de
datos_hora_limpios y aplica una interpolación lineal
86         #a los valores que son NaN. Posterior a ello, rellena esos
NaN calculando valores que "se alinean" entre los puntos
conocidos antes y después del NaN.
87
88         # Guardamos estos datos limpios
89         datos_limpios = pd.concat([datos_limpios, datos_hora_limpios
])#unimos en un solo DataFrame los últimos dos DataFrame.
90
91         # Calculamos las estadísticas con estos datos limpios
92         media_corriente = datos_hora_limpios[column].mean()
93         desviacion_std = datos_hora_limpios[column].std(ddof=0)
94         varianza = datos_hora_limpios[column].var(ddof=0)
95
96         print(f"Hora {hora}: Media = {media_corriente:.2f}, Desv.
Estándar = {desviacion_std:.2f}, Varianza = {varianza:.2f}")
97
98         # Percentiles de datos limpios
99         percentile_low_limpio = datos_hora_limpios[column].quantile
(0.001)
100         percentile_high_limpio = datos_hora_limpios[column].quantile
(0.999)
101
102         # Gráfica de datos limpios
103         puntos = ax.plot(datos_hora_limpios["t"], datos_hora_limpios
[column], marker='o', linestyle='--')
104         items_interactivos.extend(puntos)
105
106
```

```
107     # Gráfica de outliers. Siqueremos ver el puntito, solo hay
108     #if not outliers_hora.empty:
109     #ax.scatter(outliers_hora['t'], outliers_hora[column],
110     color='black', s=50, label='Outliers') #if not labels else ""
111
112     if not labels:
113         ax.hlines(y=media_corriente, xmin=t_inicio, xmax=t_fin,
114 color='gray', linestyle='--', alpha=0.7, label="Media")
115         ax.hlines(y=media_corriente + desviacion_std, xmin=
116 t_inicio, xmax=t_fin, color='red', linestyle='dotted', alpha=0.7,
117 label="Media + ")
118         ax.hlines(y=media_corriente - desviacion_std, xmin=
119 t_inicio, xmax=t_fin, color='red', linestyle='dotted', alpha=0.7,
120 label="Media - ")
121         ax.hlines(y=percentile_low_limpio, xmin=t_inicio, xmax=
122 t_fin, color='purple', linestyle='dashed', alpha=0.7, label=f"
123 Percentil {low*100}% ")
124         ax.hlines(y=percentile_high_limpio, xmin=t_inicio, xmax=
125 t_fin, color='purple', linestyle='dashed', alpha=0.7, label=f"
126 Percentil {high*100}% ")
127         labels = True
128     else:
129         ax.hlines(y=media_corriente, xmin=t_inicio, xmax=t_fin,
130 color='gray', linestyle='--', alpha=0.7)
131         ax.hlines(y=media_corriente + desviacion_std, xmin=
132 t_inicio, xmax=t_fin, color='red', linestyle='dotted', alpha=0.7)
133         ax.hlines(y=media_corriente - desviacion_std, xmin=
134 t_inicio, xmax=t_fin, color='red', linestyle='dotted', alpha=0.7)
135         ax.hlines(y=percentile_low_limpio, xmin=t_inicio, xmax=
136 t_fin, color='purple', linestyle='dashed', alpha=0.7)
137         ax.hlines(y=percentile_high_limpio, xmin=t_inicio, xmax=
138 t_fin, color='purple', linestyle='dashed', alpha=0.7)
139
140     # Interactividad con el cursor
141     cursor = mplt.cursor(items_interactivos, hover=True)
142     cursor.connect("add", lambda sel: sel.annotation.set_text(f"{sel
143 .target[1]:.2f}"))
144
145     # Etiquetas
146     plt.xticks(rotation=45)
147     ax.set_xlabel("Tiempo (Hora:Minuto:Segundo)")
148     ax.set_ylabel("Corriente (I)")
149     ax.set_title(f"Corriente {column} - 20230218 ")
150     ax.legend()
151     ax.grid(True)
152     plt.tight_layout()
```

```
138 plt.show()
139 #Checar comentarios abajo
140 outliers_export = outliers_global[['t', 'Variable']].copy()
141 outliers_export['Valor_outlier'] = outliers_global.apply(lambda row:
    row[row['Variable']], axis=1)
142 outliers_export.to_excel('outliers_detectados.xlsx', index=False)
143 print("done")
144
145
146 #Algunas cosas para recordar sobre la automatización de guardar los
    outliers en un excel.
147 #Nuestro DataFrame outliers_global contiene información sobre los
    outliers detectados en la columna "Variable" que previamente habí
    amos creado
148 #entonces, para solo guardar en el Excel un solo DataFrame con
    columnas que contengan la hora detectada del outlier, el tiempo y
    la variable,
149 #extraemos dos columnas en específico del DataFrame outliers_global
    y después hacemos una función que nos diga "Oye fila, en qué
    columna está tu outlier?"
150 #"lambda" es una forma muy corta de escribir la función y lo que
    hace "row[row['Variable']]" es, busca en cada fila, la variable
    donde está registrado el outlier
151 #y obtiene al mismo tiempo ese valor, finalmente, .apply necesita
    saber si va a aplicar la función "lambda" por fila o por columna,
    por lo que, "axis=1" aplica la función a cada fila
152 #caso contrario, axis=0 se aplicaría a cada columna. Esta nueva
    columna de "outliers_export['Valor_outlier']" ya fue añadida a
    outliers_export.
153 #La última línea convierte el archivo a Excel.
```